



Cardiff
Metropolitan
University

Prifysgol
Metropolitan
Caerdydd

School of Technologies

Module Code

Module Title

CIS6003

Advanced Programming

Academic Year

Semester

2025

1

Online room reservation System

Dimuthu Pramuditha Dinendra

Cardiff Id: st20302350

Student ID(ICBT): CL/BSCSE-CMU/06/22

Table of Contents

1. Introduction	7
1.1. Objectives of the System	7
1.2. Scope of the Project	7
2. System Analysis	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	7
3. System Design	8
3.1 Use Case Diagram	8
3.1.1 Use Case Description	8
3.1.2 <<include>> and <<extend>> Relationships	9
3.1.3 Design Justification	10
3.2 Class Diagram	11
3.2.1 Class Identification	11
3.2.2 Attributes and Methods	11
3.2.3 Relationships (Association, Aggregation, Composition)	12
3.2.4 Design Evaluation	13
3.3 Sequence Diagrams	14
3.3.1 Add New Reservation	14
3.3.2 Calculate and Print Bill	14
3.3.3 User Login Process	14
3.3.4 Critical Evaluation of the Overall Design	16
4. System Development and Architecture	17
4.1 Three-Tier Architecture Implementation	17
4.1.1 Presentation Layer	17
4.1.2 Business Logic Layer	17
4.1.3 Data Access Layer	18
4.2 Database Design (MongoDB Implementation)	18
4.2.1 Collection Structures	18
4.2.2 Primary and Foreign Key Handling in MongoDB	20
4.2.3 Advanced Database Features (MongoDB)	20
4.3 User Interface Design	20
4.3.1 Login Interface	21
4.3.2 Reservation Management Interface	23
4.3.3 Billing Interface	23
4.3.4 Display Reservation Details Interface	25

4.3.5	Help Section Interface	25
4.4	Validation Mechanisms and Error Handling	26
4.5	Application of Design Patterns	26
4.5.1	Singleton Pattern.....	26
4.5.2	MVC Pattern (Model View Controller)	26
4.5.3	DAO Pattern (Data Access Object).....	27
4.6	Critical Evaluation of Design Patterns and Their Impact	27
5.	Testing	27
5.1	Test-Driven Development (TDD) Approach	28
5.2	Test Automation Implementation	28
5.3	Test Plan	32
5.4	Test Cases	34
5.5	Requirement Traceability Matrix (RTM).....	69
5.6	Testing HTTP API Endpoints Using Postman.....	70
5.6.1	Login API Testing.....	70
5.6.2	Register API Testing	70
5.6.3	Add Reservation API Testing	71
5.6.4	Add New Room API Testing	72
5.7	Evaluation of Testing Outcomes	73
6.	Version Control and Deployment	74
6.5	Git and GitHub Repository Setup.....	74
6.6	Version Control Techniques Used.....	74
6.7	CI/CD Workflow Implementation	75
6.8	Deployment Process	76
6.9	Repository Link and Access Details	77
6.9.1	Repository Link	77
7.	Discussion and Critical Reflection	78
7.1	Strengths of the System	78
7.2	Challenges Faced During Development	78
7.3	Future Enhancements	78
8	Conclusion.....	78
	References.....	80

Table of Figures

Figure 1 Use Case Diagram.....	10
Figure 2 Class Diagram	13
Figure 3 Sequence Diagrams	15
Figure 4 Users Collection	18
Figure 5 Rooms Collection	19
Figure 6 Reservations Collection	19
Figure 7 Receptionists Collection.....	19
Figure 8 Managers Collection	20
Figure 9 Login Interface	21
Figure 10 Register Interface	22
Figure 11 Reservation Management Interface.....	23
Figure 12 Billing Interface	24
Figure 13 Display Reservation Details Interface	25
Figure 14 help section interface	25
Figure 15 UserMongoDAOTest	29
Figure 16 RoomsDAOTest.....	29
Figure 17 ReceptionistDAOTest	30
Figure 18 ReservationDAOTest	30
Figure 19 ManagerDAOTest	31
Figure 20 Test Plan	33
Figure 21Test Case 01	34
Figure 22 Test Case 02	35
Figure 23 Test Case 03	36
Figure 24 Test Case 04	37
Figure 25 Test Case 05	38
Figure 26 Test Case 06	39
Figure 27 Test Case 07	40
Figure 28 Test Case 08	41
Figure 29 Test Case 09	42
Figure 30 Test Case 10	43
Figure 31 Test Case 11	45
Figure 32 Test Case 12	46
Figure 33 Test Case 13	47
Figure 34 Test Case 14	48
Figure 35 Test Case 15	49
Figure 36 Test Case 16	50
Figure 37 Test Case 17	51
Figure 38 Test Case 18	52
Figure 39 Test Case 19	53
Figure 40 Test Case 20	55
Figure 41 Test Case 21	56
Figure 42 Test Case 22	57
Figure 43 Test Case 23	58
Figure 44 Test Case 24	59
Figure 45 Test Case 25	60
Figure 46 Test Case 26	62

Figure 47 Test Case 27	63
Figure 48 Test Case 28	65
Figure 49 Test Case 29	67
Figure 50 Test Case 30	68
Figure 51 Requirement Traceability Matrix (RTM)	69
Figure 52 Login API Testing.....	70
Figure 53 Register API Testing	71
Figure 54 Add Reservation API Testing.....	72
Figure 55 Add New Room API Testing	73
Figure 56 Meaningful Commit Messages	74
Figure 57 Branch-Based Development	75
Figure 58 Version Tagging	75
Figure 59 Continuous Integration (CI).....	76

1. Introduction

Ocean View Resort in Galle currently manages its room reservations and guest records manually. This traditional approach makes it difficult to track room availability in real time, maintain accurate guest information, and generate bills efficiently. Therefore, there is a clear need for a modern, automated system that can support secure, fast, and error free reservation management.

1.1. Objectives of the System

The main objectives of the proposed Online Room Reservation System are:

- To design and develop a secure login system for authorized users.
- To automate the process of registering guests and managing room reservations.
- To store and retrieve reservation data accurately using a database.
- To calculate and generate customer bills based on stay duration and room type.
- To provide a user-friendly interface with clear messages and validations.
- To generate useful reports to support hotel management decision-making.

1.2. Scope of the Project

This project focuses on developing a Java-based distributed room reservation system with web services and database support for Ocean View Resort. The system will cover core functionalities such as user authentication, guest registration, room booking, billing, and help services. UML diagrams, design patterns, three-tier architecture, test-driven development, and version control using GitHub will be applied in accordance with industry standards.

2. System Analysis

System analysis involves identifying and defining what the system should do and the constraints under which it must operate. This section presents the functional requirements, non-functional requirements, and key assumptions of the proposed system.

2.1 Functional Requirements

- The system shall provide the following functionalities:
- User authentication using username and password.
- Registration of new guests with unique reservation numbers.
- Storage of guest details including name, address, contact number, room type, check-in date, and check-out date.
- Viewing and searching reservation details.
- Calculation of total bill based on room type and duration of stay.
- Generation of printable invoices and summary reports.
- Help section for system usage guidelines.
- Secure exit from the system.

2.2 Non-Functional Requirements

- The system shall satisfy the following quality attributes:
- Usability: User-friendly interfaces with clear menus and messages.
- Performance: Fast response time for searching and saving reservation data.

- Security: Secure login, access control, and protection of sensitive customer information.
- Reliability: Accurate data storage with minimal system failures.
- Scalability: Ability to support an increasing number of users and reservations.
- Maintainability: Modular design using object-oriented principles and design patterns.
- Portability: Ability to run on different operating systems supporting Java.

3. System Design

The Ocean View Resort Room Reservation System is designed using Object-Oriented Design (OOD) principles. Object-oriented design focuses on modelling real-world entities as objects, encapsulating data and behaviour within classes. This approach enhances modularity, maintainability, scalability, and reusability. (GeeksforGeeks, 2018)

Key object-oriented principles applied in this system include:

- Encapsulation – Data members such as reservation details and room status are declared private and accessed through public getter and setter methods.
- Abstraction – Complex operations such as bill calculation and authentication are hidden behind service classes.
- Inheritance – Different user types (Guest, Receptionist, Manager) inherit common properties from a base User class.
- Polymorphism – Role-based functionalities are implemented using method overriding where necessary

3.1 Use Case Diagram

The Use Case Diagram represents system functionality from the perspective of different users interacting with the system. (Lucidchart, 2025)

Identification of Actors - The system contains three primary actors: Guest, Receptionist, Manager Each actor has role-based access control and distinct responsibilities within the system.

3.1.1 Use Case Description

Guest Use Cases

- Register as Guest
- Add New Reservation
- Display Reservation Details
- Display Room Details

Receptionist Use Cases

- Add New Reservation
- Manage Room Availability
- Manage Guest Accounts (Delete Guest)
- Check-in / Check-out

Manager Use Cases

- Add New Room
- Add New Receptionist
- Manage Room Availability
- Delete Guest/Receptionist Accounts

3.1.2 <<include>> and <<extend>> Relationships

1. <<include>> Relationships

- Add New Reservation <<include>> Validate Reservation Data
- Calculate Bill <<include>> Retrieve Reservation Details
- Check-in <<include>> Verify Reservation
- Login <<include>> Validate Credentials

2. <<extend>> Relationships

- Register Guest <<extend>> Add New Reservation
- Add New Reservation <<extend>> Send Bill via Email
- Check in Check out <<extend>> Send Bill via Email

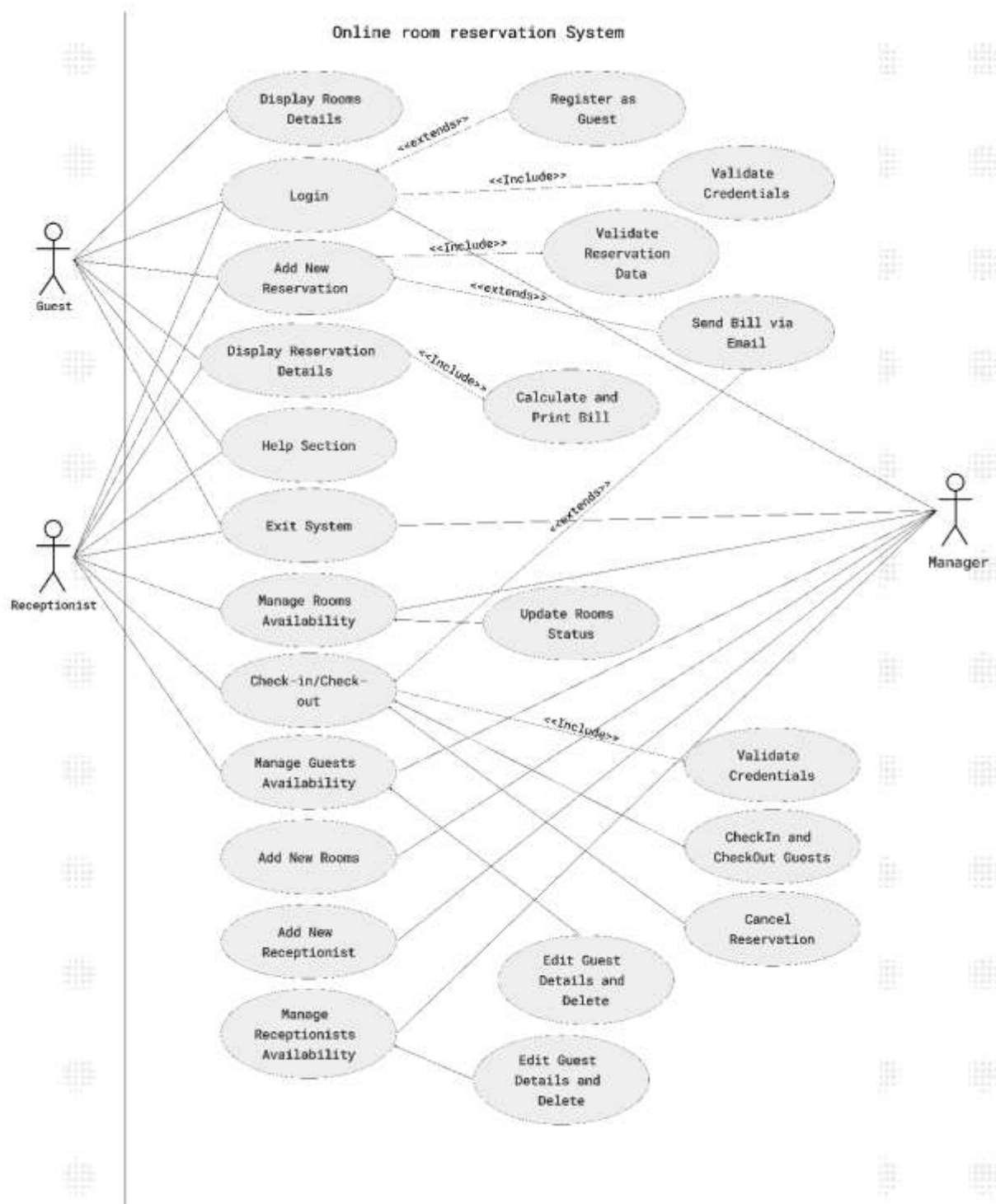


Figure 1 Use Case Diagram

3.1.3 Design Justification

The use case design ensures:

- Clear separation of responsibilities
- Role-based access control
- Reduced system conflicts
- Secure authentication before system access

The inclusion relationships reduce duplication, while extend relationships improve modularity and flexibility.

3.2 Class Diagram

The Class Diagram represents system structure, attributes, methods, and relationships. (GeeksforGeeks, 2018)

3.2.1 Class Identification

Main classes:

- Guest
- Receptionist
- Manager
- Reservation
- Room
- Bill
- RoomService
- ReservationService
- EmailService

3.2.2 Attributes and Methods

User (abstract):

- Attributes: username (String), password (String).
- Methods: login(username, password): boolean; logout(): void.

Guest (extends User):

- Attributes: name (String), address (String), contactNumber (String).
- Methods: register(name, address, contact, username, password): void
addReservation(...): Reservation displayReservationDetails(reservationId): void
displayRooms(): void getHelp(): String; exit(): void.

Receptionist (extends User):

- Attributes: employeeId (String), shift (String).
- Methods: addReservation(...): Reservation manageRoomsAvailability(roomId, status): void
manageGuests(guestId, action): void checkIn(reservationId): void
checkOut(reservationId): void calculateBill(reservationId): Bill getHelp(): String
exit(): void.

Manager (extends User):

- Attributes: (inherits from User).
- Methods: addRoom(...): Room; addReceptionist(...): Receptionist
manageRoomsAvailability(roomId, status): void; manageGuests(guestId, action): void
manageReceptionists(receptionistId, action): void; calculateBill(reservationId): Bill
exit(): void.

Reservation:

- Attributes: reservationNumber (int), guest (Guest), room (Room), checkInDate (Date), checkOutDate (Date).
- Methods: createReservation(...): void; getDetails(): String.

Room:

- Attributes: roomNumber (int), type (String), floor (int), maxGuests (int), pricePerNight (double), status (String: available/cleaning/occupied), amenities (List<String>), description (String).
- Methods: updateStatus(status): void getDetails(): String.

Bill:

- Attributes: reservation (Reservation), totalCost (double).
- Methods: calculate (nights, rate): double; printBill(): void.

Email Service:

- Attributes: smtpHost (String), smtpPort (String), senderEmail (String)
- Method: boolean sendbill (toEmail, bill), Boolean sendconfirmation (toemail, res), void billMessage(bill)

3.2.3 Relationships (Association, Aggregation, Composition)

- Association: Loose links, e.g., Guest associates with Reservation (Guest makes Reservation).
- Aggregation: "Has-a" where parts can exist independently, e.g., Manager aggregates Receptionist (Manager manages Receptionists, but Receptionists exist without Manager).
- Composition: "Owns-a" where parts cannot exist without whole, e.g., Reservation composes Bill (Bill is part of Reservation lifecycle).

Access Modifiers (Private/Public)

- Private: Sensitive data like password (-password: String).
- Public: Accessible methods like + login(): boolean.

- Protected: For inheritance, e.g., #username: String.

Multiplicity and Navigability

- Multiplicity: Guest 1..* - Reservation (one guest can have multiple reservations); Room 1 - Reservation 0..1 (one room per reservation, optional).
- Navigability: Arrows indicate direction, e.g., Receptionist - Room (Receptionist navigates to update Room, but not vice versa).

3.2.4 Design Evaluation

The class diagram uses inheritance to avoid code duplication (e.g., shared login in User). Relationships ensure data integrity (e.g., composition prevents orphaned bills). Access modifiers enhance security. Multiplicity models real constraints (e.g., one room per booking). Overall, it's robust but could be extended with interfaces for polymorphism if more roles are added.

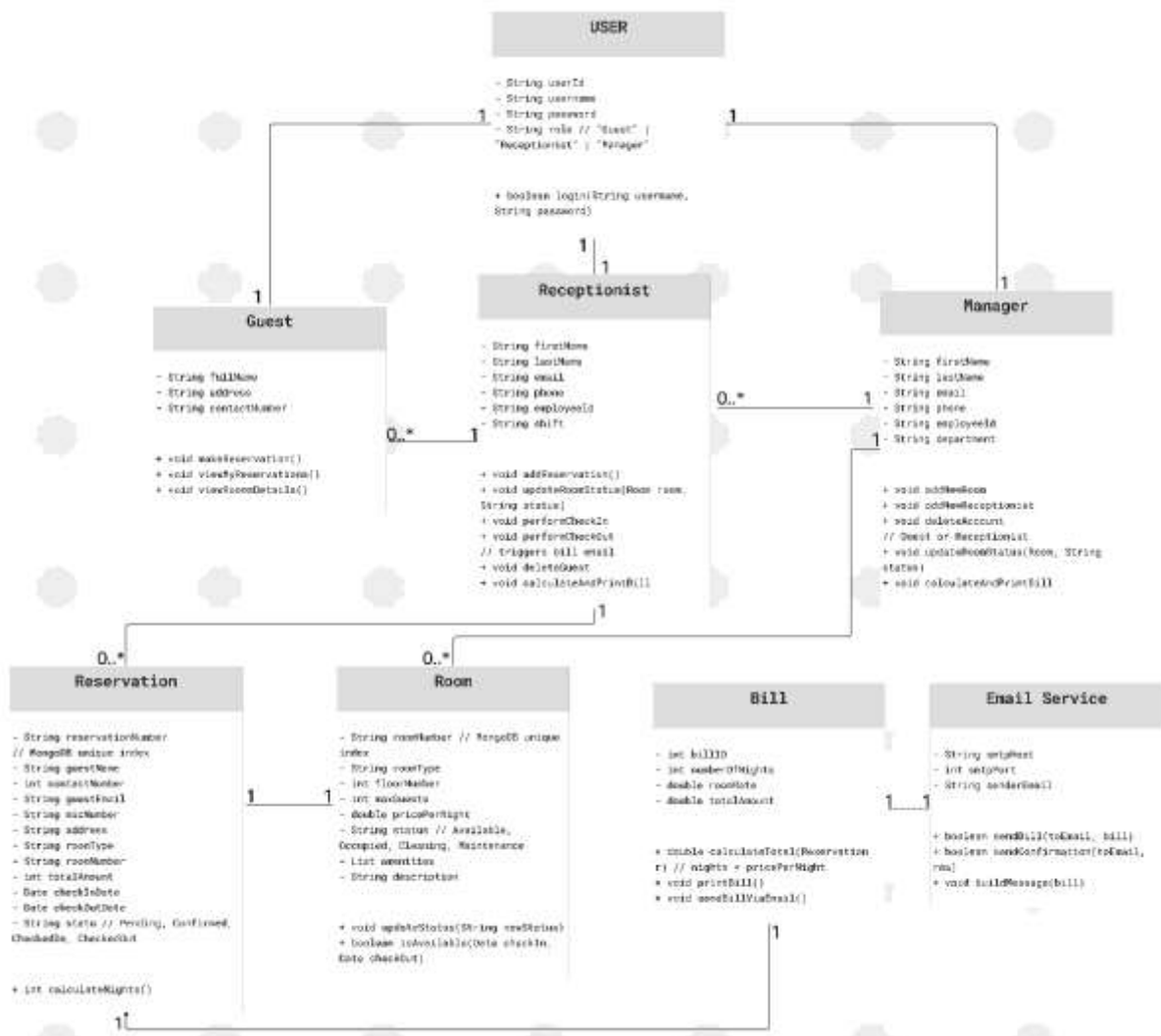


Figure 2 Class Diagram

3.3 Sequence Diagrams

Sequence diagrams illustrate the dynamic interactions between objects in the hotel reservation system over time, focusing on message exchanges for key scenarios. They use UML notation to show lifelines (representing actors and objects), messages (synchronous calls with arrows), and control structures like alternatives (alt) for conditional flows. (Lucidchart, 2019)

3.3.1 Add New Reservation

This diagram models the process for a Guest or Receptionist creating a new booking. It involves checking room availability, creating the reservation, and updating room status. Error handling is included for unavailable rooms.

- User - Login
- User - ReservationManager : createReservation()
- ReservationManager - RoomManager : checkAvailability()
- RoomManager - Database : fetchRoom()
- Database - RoomManager : return data
- ReservationManager - Database : saveReservation()
- System - User : confirmation message

3.3.2 Calculate and Print Bill

This diagram depicts computing the bill based on reservation details, applicable to Guest, Receptionist, or Manager. It fetches data, calculates costs, and outputs the bill.

- User - ReservationManager : getReservation()
- Reservation - calculateNights()
- Send Email - sendBillViaEmail(guestEmail, bill)
- Bill - calculateBill()
- System - printBill()
- Total displayed to user

3.3.3 User Login Process

This diagram covers authentications for all actors (Guest, Receptionist, Manager), validating credentials and granting role-based access.

- User enters username & password
- AuthenticationService validates credentials
- System checks database
- Access granted or denied

Manager Receptionist Guest

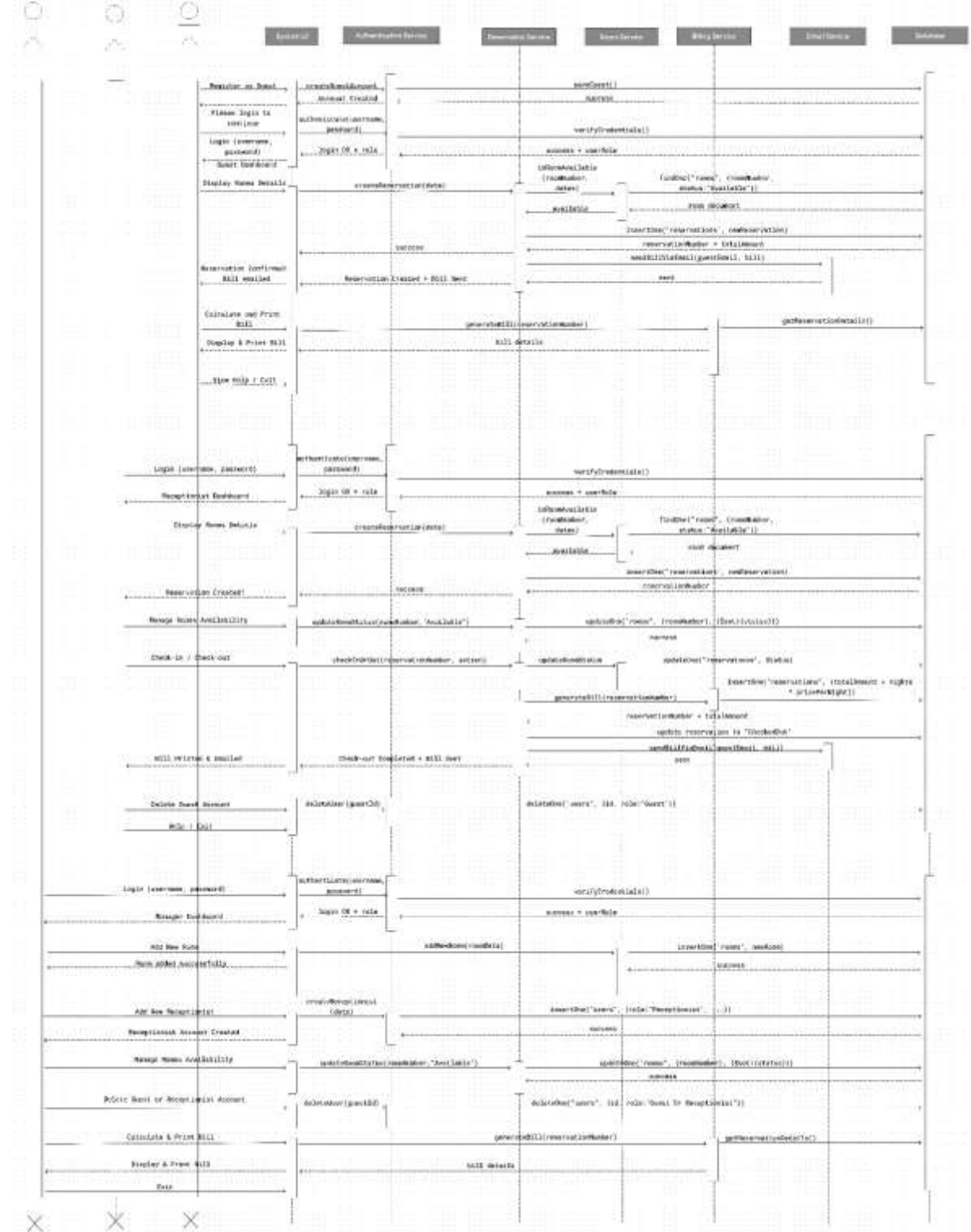


Figure 3 Sequence Diagrams

3.3.4 Critical Evaluation of the Overall Design

The proposed UML design effectively applies object-oriented principles to model real-world hotel operations in a structured and logical manner. The system demonstrates a clear role-based architecture, separating Guest, Receptionist, and Manager responsibilities to ensure proper access control and security. The authentication mechanism enhances system protection, while the logical separation of entities such as User, Room, Reservation, and Bill promotes modularity and maintainability.

However, the current design assumes a single hotel branch and does not incorporate distributed or microservices architecture at the UML level. Additionally, an external payment gateway integration is not included. Overall, the design provides a balanced, secure, and maintainable foundation aligned with industry standards and academic expectations for high achievement.

4. System Development and Architecture

The Ocean View Resort Reservation System is implemented using a structured three-tier architecture, ensuring separation of concerns, scalability, and maintainability. This architecture divides the system into Presentation Layer, Business Logic Layer, and Data Access Layer. Such separation improves modularity and allows independent modification of each layer without affecting the entire system.

4.1 Three-Tier Architecture Implementation

The system architecture consists of:

- Presentation Layer (UI Layer) – Handles user interaction.
- Business Logic Layer (BLL) – Processes system rules and operations.
- Data Access Layer (DAL) – Manages database communication.

This layered structure enhances security, testability, and flexibility, aligning with modern enterprise system design principles. (GeeksforGeeks, 2016)

4.1.1 Presentation Layer

The Presentation Layer represents the front-end interface of the Ocean View Resort Reservation System and is responsible for handling all user interactions. This layer acts as the communication bridge between users (Guest, Receptionist, and Manager) and the system's business logic. It is designed to be user-friendly, intuitive, and responsive while maintaining strict separation from backend processing and database operations.

The Presentation Layer is implemented using Java web technologies and follows the View component of the MVC (Model-View-Controller) architectural pattern. It collects user input, validates basic data formats, forwards requests to the Business Logic Layer, and displays processed results back to the user.

- Login screen
- Reservation management forms
- Room management dashboard
- Billing interface
- Report viewing interface

4.1.2 Business Logic Layer

The Business Logic Layer is the core component of the Ocean View Resort Reservation System. It is responsible for processing all system rules, enforcing business policies, and coordinating communication between the Presentation Layer and the Data Access Layer. This layer ensures that all operations follow hotel management rules before any data is stored or retrieved from the database.

The Business Logic Layer handles:

- Authentication validation
- Reservation creation
- Room availability checking

- Bill calculation
- Check-in and check-out operations

4.1.3 Data Access Layer

The Data Access Layer (DAL) is responsible for managing all interactions between the system and the database. It acts as an intermediary layer that isolates database operations from the Business Logic Layer, ensuring that data persistence logic is separated from business rules. This separation improves maintainability, flexibility, and scalability of the system.

In the Ocean View Resort Reservation System, the Data Access Layer is implemented using the DAO (Data Access Object) pattern, where each main entity has a dedicated DAO class. These include:

- NewReservationDAO
- ReceptionistDAO
- RoomsDAO
- UserMongoDAO

4.2 Database Design (MongoDB Implementation)

The Ocean View Resort Reservation System uses MongoDB, a NoSQL document-oriented database, to store and manage system data. MongoDB was selected due to its flexibility, scalability, and ability to handle structured and semi-structured data efficiently. Unlike relational databases, MongoDB stores data in JSON-like documents (BSON format), making it suitable for dynamic hotel management data such as room amenities, guest details, and reservations.

The use of MongoDB supports faster development, easier schema evolution, and better performance for distributed applications and web services.

4.2.1 Collection Structures

1. Users Collection

```
_id: ObjectId('69824196ab616478c3cc1540')
firstName: "Dimuthu"
lastName: "Pramuditha"
email: "dimuthu01@gmail.com"
phoneNumber: "0778889999"
role: "Guest"
password: "ef797c8118f02dfb649607dd5d3f8c7623048c9c063d532cc95c5ed7a898a64f"
createdAt: "2026-02-04T00:12:30.594702500"
updatedAt: "2026-02-28T20:51:12.865444200"
```

Figure 4 Users Collection

2. Rooms Collection

```
_id: ObjectId('69883d357389b73b04011190')
roomNumber: "100"
roomType: "Double"
floorNumber: "3"
maxOccupancy: "5"
pricePerNight: "12700"
availabilityStatus: "Occupied"
amenities: "WiFi, AC, TV, Balcony"
description: "The room features a clean, modern design with neutral tones and simple..."

_id: ObjectId('698c2e2a713ac82b454a32b9')
roomNumber: "101"
roomType: "Deluxe"
floorNumber: "3"
maxOccupancy: "6"
pricePerNight: "15800"
availabilityStatus: "Maintenance"
amenities: "WiFi, AC, TV, MiniBar, Balcony, Jacuzzi"
description: "Enjoy extra space and comfort in our Deluxe Room, featuring premium fu..."
```

Figure 5 Rooms Collection

3. Reservations Collection

```
_id: ObjectId('69a14f2769ec2f2abf07fd80')
reservationNumber: "RES-2026-001"
guestName: "Dimuthu Pramudithu"
contactInfo: "0798634578"
nicNumber: "901234567V"
address: "Kirindiwela, Gampaha"
roomType: "Double"
roomNumber: "100"
totalAmount: 30175.2
checkInDate: "2026-02-27T14:00"
checkOutDate: "2026-03-01T15:30"
status: "Checked In"
userEmail: "receptionist@hotel.com"
```

Figure 6 Reservations Collection

4. Receptionists Collection

```
_id: ObjectId('69a01c74d72f527c060f78b0')
firstName: "Dimuthu"
lastName: "Pramuditha"
email: "receptionist@hotel.com"
phoneNumber: "0718674583"
password: "a27dce9d8b5488238487ca36967563b7487b12232e3d1cb98442360f833cfbd7"
employeeId: "EMP001"
shift: "Morning"
role: "Receptionist"
createdAt: "2026-02-26T15:42:04.476913200"
updatedAt: "2026-02-27T11:33:32.311610600"
```

Figure 7 Receptionists Collection

5. Managers Collection

```
_id: ObjectId('69961321a8ae24391b4728d7')
firstName: "Dimuthu"
lastName: "Pramuditha"
email: "manager@hotel.com"
phoneNumber: "0785423691"
password: "866485796cfa8d7c0cf7111640205b83076433547577511d81f8030ae99ecea5"
employeeId: "MGR-2026-001"
department: "Operations"
role: "Manager"
createdAt: "2026-02-18T19:29:37.933Z"
updatedAt: "2026-02-18T19:29:37.939Z"
```

Figure 8 Managers Collection

4.2.2 Primary and Foreign Key Handling in MongoDB

MongoDB does not use traditional primary and foreign keys like relational databases. Instead:

- `_id` acts as the primary identifier
- `ObjectId` references act as logical foreign keys
- Relationships are managed at application level

4.2.3 Advanced Database Features (MongoDB)

To enhance performance, reliability, and scalability, the system uses several advanced MongoDB features

- Triggers automatically update room status to “Occupied” booking the room.
- Triggers automatically update rooms status to Cleaning after check-out each room
- Stored procedures are used to calculate total bill.
- Constraints prevent negative pricing or invalid date entries.

Using MongoDB provides high flexibility and scalability compared to traditional relational databases. The document-based model allows dynamic handling of room amenities and user roles without complex joins. However, because MongoDB does not enforce foreign key constraints automatically, careful application-level validation is required to maintain referential integrity.

Overall, the MongoDB implementation supports distributed architecture, enhances performance, and aligns with modern web-based system development standards.

4.3 User Interface Design

The User Interface (UI) of the Ocean View Resort Reservation System is developed using JSP (JavaServer Pages) and Servlets, following a Model-View-Controller (MVC) architecture. JSP is used to design dynamic web pages (View), Servlets act as Controllers to process requests, and MongoDB is accessed through JDBC style database connectivity libraries and DAO classes for data persistence.

This architecture ensures clear separation between presentation logic and business logic while supporting scalability and maintainability.

- JSP - Handles frontend presentation
- Servlets - Handle HTTP requests and responses
- Business Logic Classes - Process operations
- DAO Layer (JDBC-based implementation) - Communicate with MongoDB

Although MongoDB is a NoSQL database, JDBC-like structured database access patterns are used within DAO classes to standardize database operations

4.3.1 Login Interface

- Username and password fields
- Password masking
- Role-based redirection (Guest, Receptionist, Manager)
- Error message display for invalid login

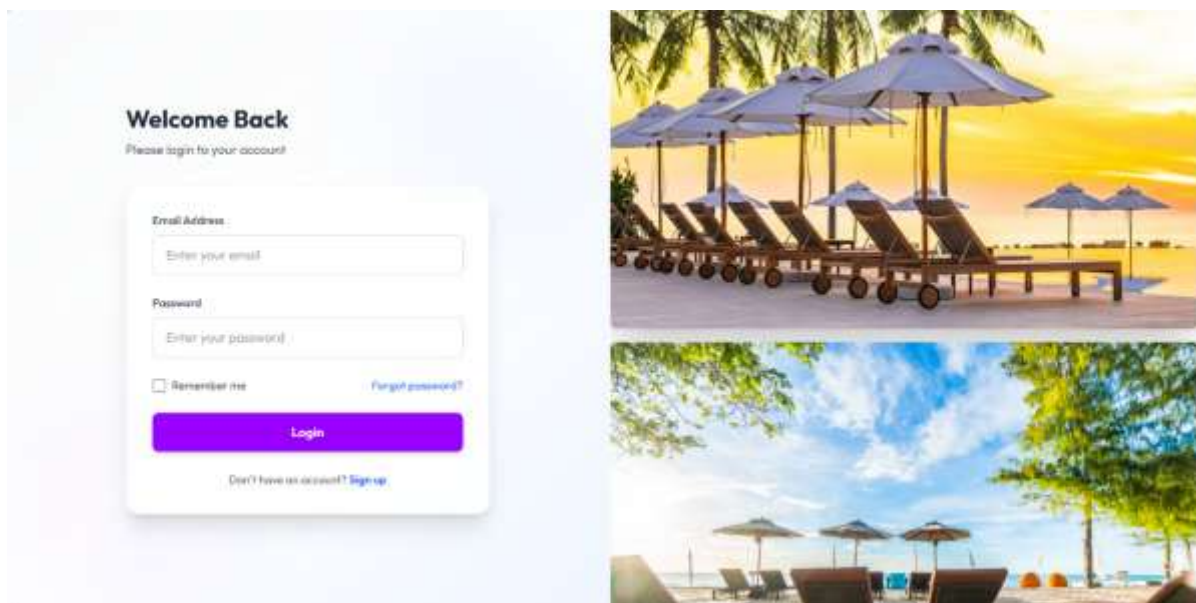


Figure 9 Login Interface

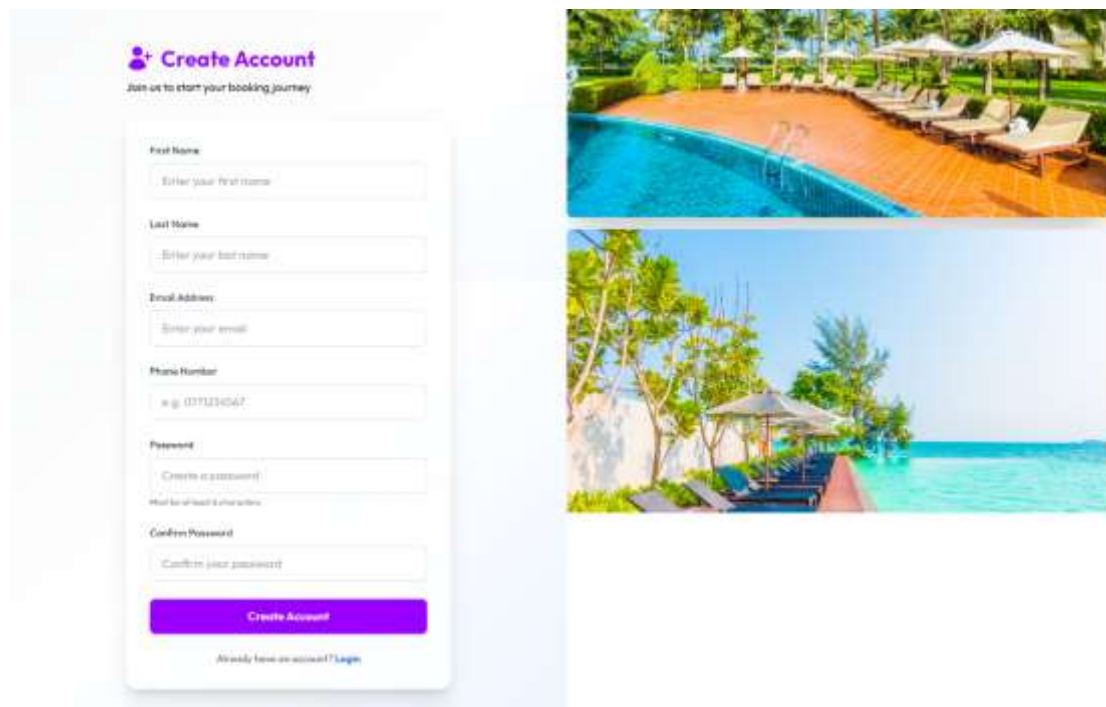


Figure 10 Register Interface

4.3.2 Reservation Management Interface

- Add reservation form
- Date selection
- Add Room Number
- Reservation confirmation message

The screenshot shows the 'New Reservation' form in the Ocean View Resort system. The form is divided into two main sections: 'Guest Information' and 'Reservation Details'. The 'Guest Information' section includes fields for 'Guest Name' (with a placeholder 'Enter guest full name'), 'Contact Number' (with a placeholder '+1 (555) 123-4567'), 'REC Number' (with a placeholder 'e.g. 99902345678 or 902345678'), and 'Address' (with a placeholder 'Enter guest address'). The 'Reservation Details' section includes a 'Reservation Number' field (with a placeholder 'Will be assigned on submission (e.g. RES-2026-001)' and a note 'The reservation number is assigned automatically by the system after successful file save'), a 'Room Number' field (with a placeholder 'Select available room'), and 'Check-in Date' and 'Check-out Date' fields (both with placeholders 'dd/mm/yyyy' and calendar icons). At the bottom right of the form are 'Cancel' and 'Create Reservation' buttons. The left sidebar shows the 'New Booking' button and a list of menu items: Dashboard, Reservations, Rooms, Billing, Settings, Help Center, and Logout. The user profile at the bottom left is 'Dinetha Premaditha'.

Figure 11 Reservation Management Interface

4.3.3 Billing Interface

The Billing Interface is designed to generate, display, and print the total cost of a guest's stay in a clear and user-friendly manner. It is implemented using JSP for the frontend (View) and a dedicated BillingServlet (Controller) that communicates with the Business Logic Layer and MongoDB through DAO classes.

The primary purpose of this interface is to calculate the total amount based on the number of nights stayed and the room price per night, and to present the final bill to the user (Guest, Receptionist, or Manager).

- Display stay duration
- Room rate
- Total cost calculation

- Print/export option

**Ocean View Resort**

New Booking

45360

Dashboard

Reservations

Rooms

Billing

SETTINGS

Help Center

Logout

Dimuthu Prasadithu

dimuthu@oceanviewresort.com

Owner

Search for bookings, guests, rooms...

My Reservations

View and manage all your hotel bookings

Select Invoice: -- Choose Reservation --

Print

INVOICE

Ocean View Resort

Luxury Hotel & Spa

Invoice #RES-2026-001

Date: Feb 18, 2026

Guest Details

Guest Name

Dimuthu Prasadithu

Contact Number

0798634578

Address

Kirindiwella, Gampaha

Booking Details

Reservation Number

RES-2026-001

Room Number

100

Room Type

standard

Check-in Date

Feb 18, 2026

Check-out Date

Feb 18, 2026

Number of Nights

0 Nights

Bill Summary

Description	Quantity	Rate	Amount
Room Charges	0 nights	LKR 12700.00	LKR 12700.00
standard - Room 100			
Subtotal (Room Charges):			LKR 12700.00
Service (00%)			LKR 1270.00
Tax (8%)			LKR 1017.60
TOTAL AMOUNT:			LKR 15087.60

Thank You for Your Stay!

We hope you enjoyed your experience at Ocean View Resort!

We look forward to welcoming you again soon!

Contact Information

Phone

+1 (555) 123-4567

Email

info@oceanviewresort.com

Address

123 Ocean Drive, Beach City

This is a computer-generated invoice and does not require a signature.

For any queries, please contact our reception desk.

Figure 12 Billing Interface

4.3.4 Display Reservation Details Interface

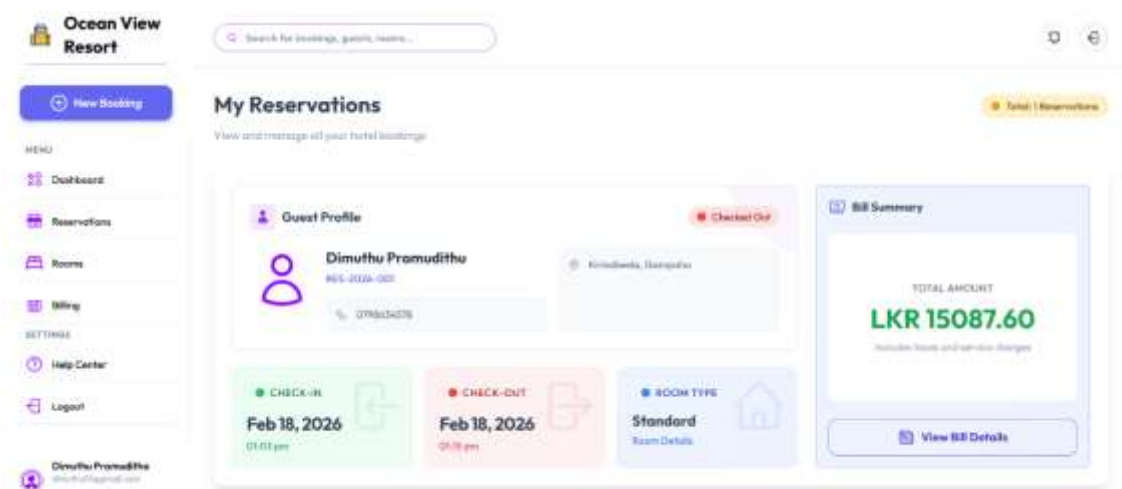


Figure 13 Display Reservation Details Interface

4.3.5 Help Section Interface

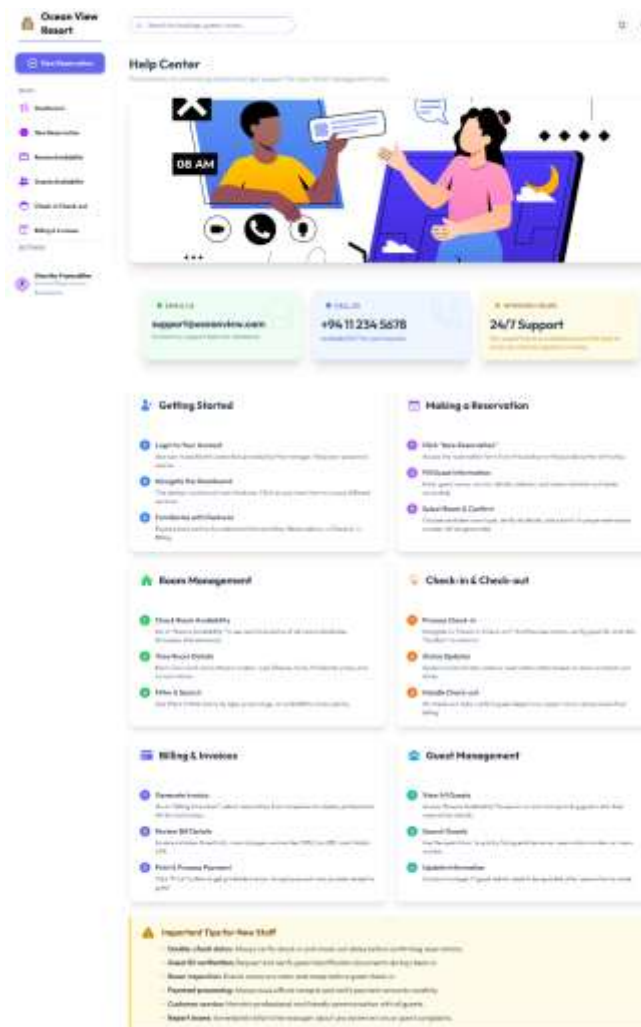


Figure 14 help section interface

4.4 Validation Mechanisms and Error Handling

Validation and error handling are critical components of the Ocean View Resort Reservation System to ensure data integrity, system reliability, and secure operation. The system implements multi-layer validation strategies across the Presentation Layer (JSP), Business Logic Layer (Services), and Data Access Layer (DAO), ensuring that incorrect or malicious inputs are detected and handled appropriately.

- Input validation (non-empty fields, valid dates)
- Password strength validation
- Numeric validation for pricing
- Exception handling using try-catch blocks
- User-friendly error messages

4.5 Application of Design Patterns

The Ocean View Resort Reservation System applies several well-established design patterns to enhance modularity, scalability, maintainability, and code reusability. The use of design patterns ensures that the system follows industry best practices and avoids tightly coupled or poorly structured code. The primary patterns implemented in this system are Singleton, MVC, and DAO.

4.5.1 Singleton Pattern

The Singleton pattern is used to manage the database connection instance. Since multiple database connections can consume excessive resources and reduce performance, a single shared instance of the database connection is maintained throughout the application.

For example, a `MongoDBConnection` class ensures:

- Only one connection instance is created.
- All DAO classes reuse the same connection.
- Resource usage is optimized.
- Thread safety is maintained.

This pattern improves performance and prevents unnecessary object creation, particularly in a web-based environment where multiple requests are handled simultaneously.

4.5.2 MVC Pattern (Model View Controller)

The Ocean View Resort Reservation System is developed using the Model View Controller (MVC) architectural pattern to ensure clear separation of concerns, improved maintainability, and structured code organization. MVC divides the application into three interconnected components: Model, View, and Controller. This separation enhances scalability and allows independent development and modification of each layer. (GeeksforGeeks, 2017)

- Model - Business classes and MongoDB entities (Reservation, Room, User, Bill)
- View - JSP pages (login.jsp, reservation.jsp, billing.jsp)
- Controller - Servlets (LoginServlet, ReservationServlet, BillingServlet)

4.5.3 DAO Pattern (Data Access Object)

The Data Access Object (DAO) pattern is implemented in the Ocean View Resort Reservation System to separate database operations from business logic. This pattern ensures that all data persistence responsibilities are handled in dedicated classes, preventing direct interaction between the Business Logic Layer and MongoDB.

By using the DAO pattern, the system achieves better modularity, maintainability, and scalability.

Each main entity has a dedicated DAO class such as:

- ManagerDAO
- NewReservationDAO
- ReceptionistDAO
- RoomsDAO
- UserMongoDAO

The primary purpose of the DAO pattern is to:

- Encapsulate all database access code
- Isolate persistence logic from business logic
- Provide a clean API for CRUD operations
- Improve code reusability and maintainability

4.6 Critical Evaluation of Design Patterns and Their Impact

The selected design patterns significantly enhance the system's structure and maintainability. The Singleton pattern optimizes resource usage MVC ensures clear separation of concerns, making the system easier to test and extend. The DAO pattern enhances portability by isolating database operations. However, excessive pattern usage can increase system complexity. Careful implementation ensures patterns improve scalability without overengineering the solution.

Overall, the chosen patterns align with industry standards and contribute to a modular, maintainable, and extensible architecture.

5. Testing

The testing for the Ocean View Resort Reservation System was designed to ensure reliability, accuracy, security, and overall system stability. Since the system manages sensitive guest information, room availability, and billing calculations, a structured and multi-level testing approach was essential to maintain data integrity and prevent operational errors.

A layered testing strategy was adopted, covering Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT). Each layer focuses on validating different aspects of the system. Unit testing verifies individual components such as authentication, reservation processing, and billing calculations. Integration testing ensures smooth communication between the Presentation Layer (JSP and Servlets), Business Logic Layer, and

Data Access Layer (MongoDB). System testing validates the complete application workflow, including role-based access for Guests, Receptionists, and Managers.

The rationale behind this strategy is early defect detection, reduced maintenance cost, and improved software quality. By testing components individually before integrating them, logical and functional errors can be identified at an early stage. Additionally, validation and negative testing were applied to ensure the system handles invalid inputs, such as incorrect login credentials or invalid reservation dates, gracefully and securely.

Automation tools such as JUnit were used to improve efficiency and enable regression testing after every code update. This ensures that newly added features do not break existing functionality.

5.1 Test-Driven Development (TDD) Approach

Test-Driven Development (TDD) is a software development methodology where you write automated tests before you write the production code. (GeeksforGeeks, 2020) The Ocean View Resort Reservation System was developed using a Test-Driven Development (TDD) approach to enhance code quality, reliability, and maintainability. TDD is a development methodology where test cases are written before implementing the actual functionality. This ensures that every feature is validated from the beginning of the development process.

5.2 Test Automation Implementation

Test automation was implemented to improve reliability, repeatability, and efficiency of the testing process. Automated unit tests were written using Junit (JUnit Jupiter) and Mockito, following modern Java testing standards. This ensured that critical components of the system were continuously validated throughout development.

Automated tests focused mainly on the Data Access Layer to ensure correct database interaction logic. The following DAO classes were tested:

- RoomsDAO
- UserMongoDAO
- ReceptionistDAO
- ManagerDAO
- NewReservationDAO

Mockito was used to simulate MongoDB interactions without connecting to the actual database. This approach:

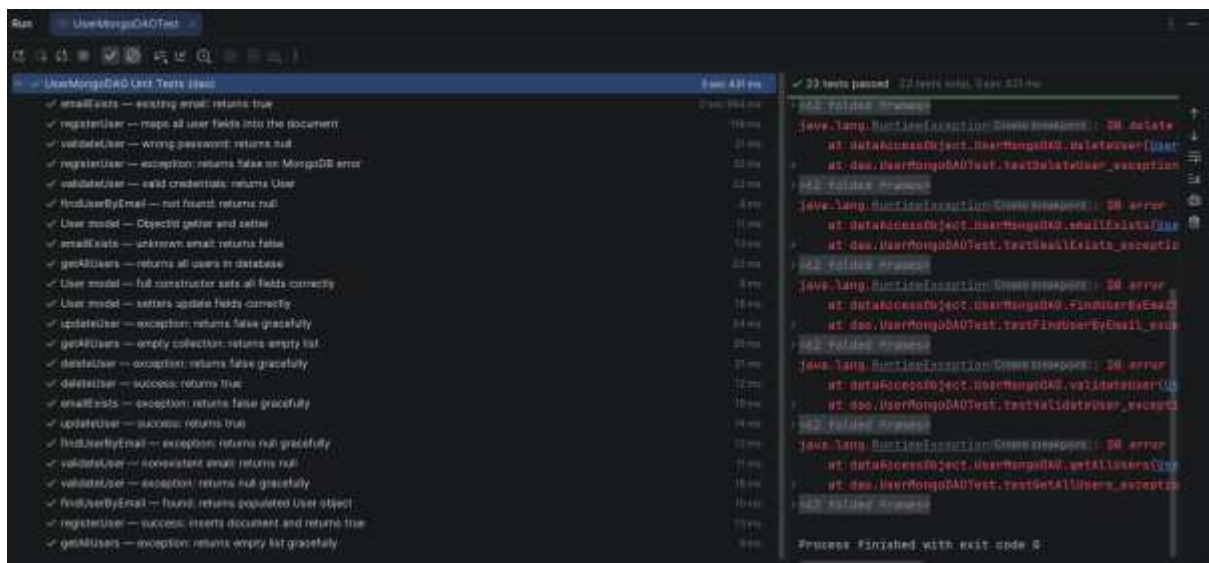
- Prevented dependency on a live MongoDB instance
- Improved test execution speed
- Enabled isolated testing of DAO logic

The use of Junit and Mockito significantly strengthened the quality assurance process. Mocking database interactions allowed independent validation of business logic without external dependencies. Although writing automated tests required additional development effort initially, it greatly improved system stability and maintainability.

Each DAO was tested for:

- Create operations (Insert data)
- Read operations (Retrieve data)
- Update operations
- Delete operations
- Validation logic
- Exception handling scenarios

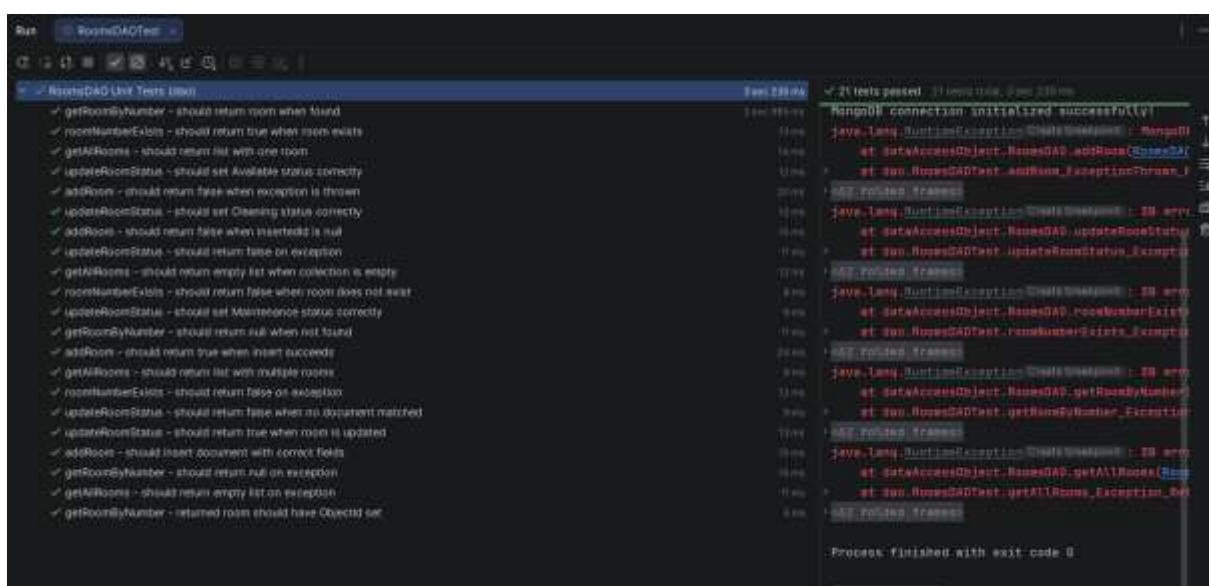
1. UserMongoDAOTest



```
Run UserMongoDAOTest
UserMongoDAO Unit Tests: 30 tests passed, 22 tests failed, 2 tests skipped
30 tests passed, 22 tests failed, 2 tests skipped
1. emailExists - existing email: returns true
2. registerUser - maps all user fields into the document
3. validateUser - wrong password: returns null
4. registerUser - exception: returns false on MongoDB error
5. validateUser - valid credentials: returns User
6. findUserByEmail - not found: returns null
7. User model - ObjectId getter and setter
8. emailExists - unknown email: returns false
9. getUsers - returns all users in database
10. User model - full constructor sets all fields correctly
11. User model - setters update fields correctly
12. updateUser - exception: returns false gracefully
13. getUsers - empty collection: returns empty list
14. deleteUser - exception: returns false gracefully
15. deleteUser - success: returns true
16. emailExists - exception: returns false gracefully
17. updateUser - success: returns true
18. findUserByEmail - exception: returns null gracefully
19. validateUser - nonexistent email: returns null
20. validateUser - exception: returns null gracefully
21. findUserByEmail - found: returns populated User object
22. registerUser - success: inserts document and returns true
23. getUsers - exception: returns empty list gracefully
24. deleteRoom - should return room when found
25. roomNumberExists - should return true when room exists
26. getRooms - should return list with one room
27. updateRoomStatus - should set Available status correctly
28. addRoom - should return false when exception is thrown
29. updateRoomStatus - should set Cleaning status correctly
30. addRoom - should return false when inserted is null
31. updateRoomStatus - should return false on exception
32. getRooms - should return empty list when collection is empty
33. roomNumberExists - should return false when room does not exist
34. updateRoomStatus - should set Maintenance status correctly
35. getRoomByNumber - should return null when not found
36. addRoom - should return true when insert succeeds
37. getRooms - should return list with multiple rooms
38. roomNumberExists - should return false on exception
39. updateRoomStatus - should return false when no document matched
40. updateRoomStatus - should return true when room is updated
41. addRoom - should insert document with correct fields
42. getRoomByNumber - should return null on exception
43. getRooms - should return empty list on exception
44. getRoomByNumber - returned room should have ObjectId set
```

Figure 15 UserMongoDAOTest

2. RoomsDAOTest



```
Run RoomsDAOTest
RoomsDAO Unit Tests: 21 tests passed, 21 tests failed, 2 tests skipped
21 tests passed, 21 tests failed, 2 tests skipped
1. MongoDB connection initialized successfully!
2. java.lang.NullPointerException: MongoDB: MongoDB
3. et: dataAccessObject.RoomsDAO.addRoom(20000)
4. et: dao.RoomsDAOTest.addRoom_exceptionThrown_1
5. java.lang.NullPointerException: MongoDB: MongoDB
6. et: dataAccessObject.RoomsDAO.updateRoomStatus
7. et: dao.RoomsDAOTest.updateRoomStatus_exceptionThrown_1
8. java.lang.NullPointerException: MongoDB: MongoDB
9. et: dataAccessObject.RoomsDAO.roomNumberExists
10. et: dao.RoomsDAOTest.roomNumberExists_exceptionThrown_1
11. java.lang.NullPointerException: MongoDB: MongoDB
12. et: dataAccessObject.RoomsDAO.getRoomByNumber
13. et: dao.RoomsDAOTest.getRoomByNumber_exceptionThrown_1
14. java.lang.NullPointerException: MongoDB: MongoDB
15. et: dataAccessObject.RoomsDAO.getAllRooms
16. et: dao.RoomsDAOTest.getAllRooms_exceptionThrown_1
17. Process finished with exit code 0
```

Figure 16 RoomsDAOTest

3. ReceptionistDAOTest

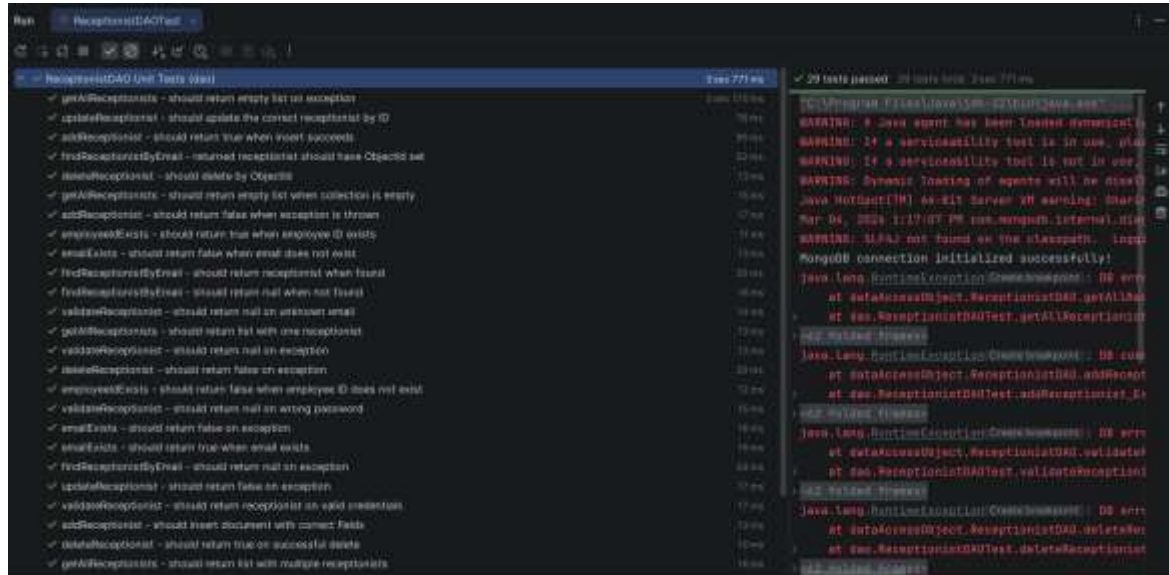


Figure 17 ReceptionistDAOTest

4. ReservationDAOTest

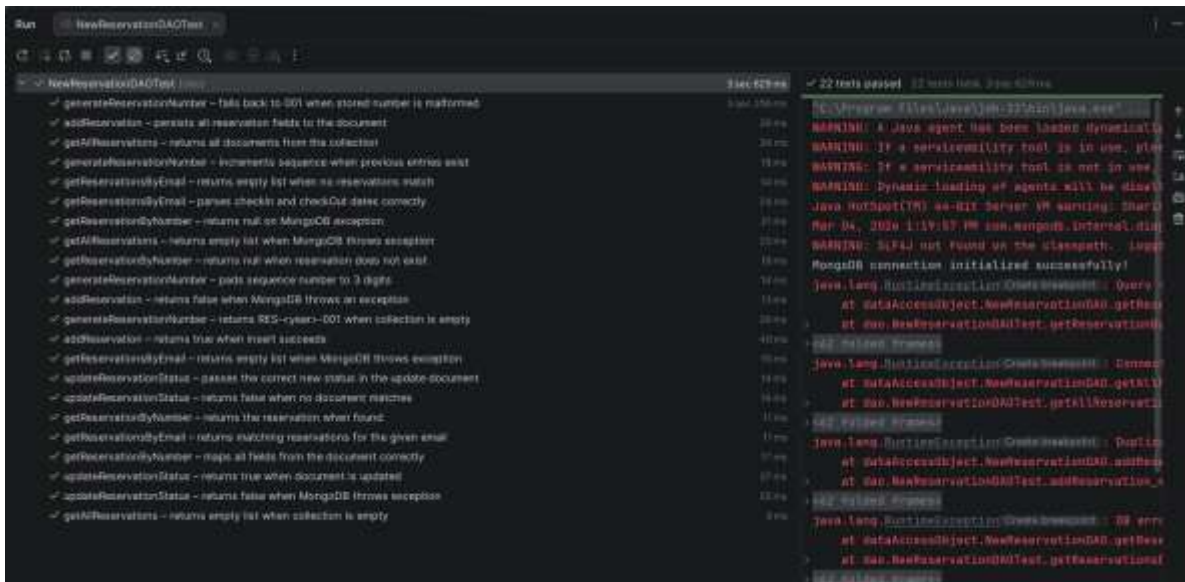
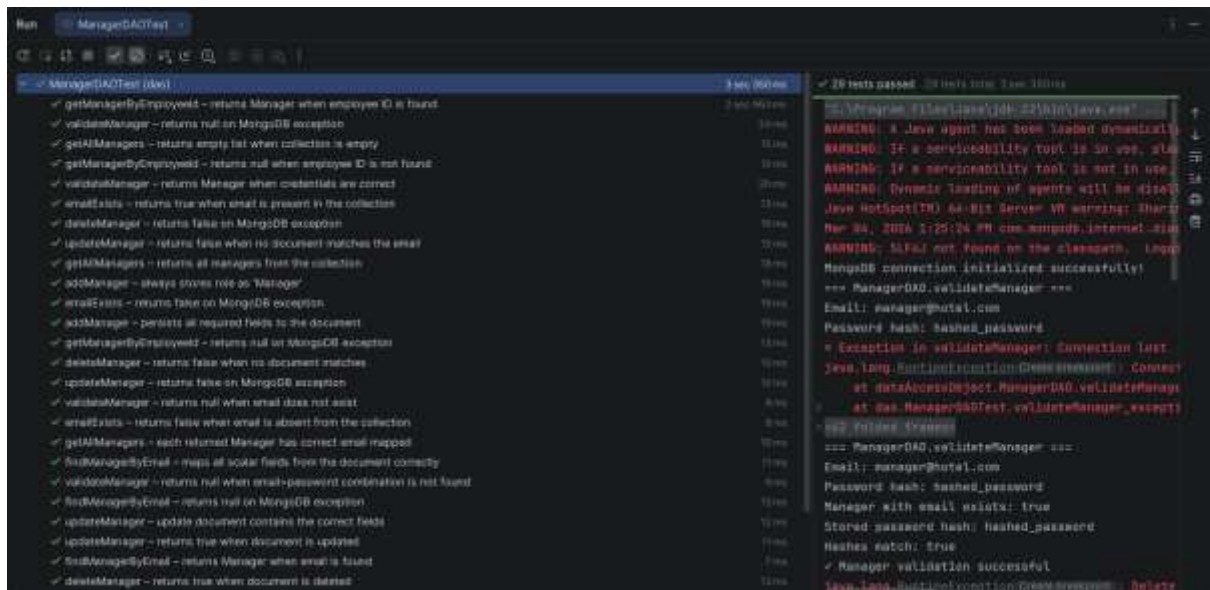


Figure 18 ReservationDAOTest

5. ManagerDAOTest



```
Run ManagerDAOTest
ManagerDAOTest (dot) 3 sec 900ms
✓ getManagerByEmployeeId - returns Manager when employee ID is found 10ms
✓ validateManager - returns null on MongoDB exception 10ms
✓ getAllManagers - returns empty list when collection is empty 10ms
✓ getManagerByEmployeeId - returns null when employee ID is not found 10ms
✓ validateManager - returns Manager when credentials are correct 10ms
✓ emailExists - returns true when email is present in the collection 10ms
✓ deleteManager - returns false on MongoDB exception 10ms
✓ updateManager - returns false when no document matches the email 10ms
✓ getAllManagers - returns all managers from the collection 10ms
✓ addManager - always stores role as 'Manager' 10ms
✓ emailExists - returns false on MongoDB exception 10ms
✓ addManager - persists all required fields to the document 10ms
✓ getManagerByEmployeeId - returns null on MongoDB exception 10ms
✓ deleteManager - returns false when no document matches 10ms
✓ updateManager - returns false on MongoDB exception 10ms
✓ validateManager - returns null when email does not exist 5ms
✓ emailExists - returns false when email is absent from the collection 5ms
✓ getAllManagers - each returned Manager has correct email mapped 10ms
✓ findManagerByEmail - maps all scalar fields from the document correctly 10ms
✓ validateManager - returns null when email-password combination is not found 10ms
✓ findManagerByEmail - returns null on MongoDB exception 10ms
✓ updateManager - update document contains the correct fields 10ms
✓ updateManager - returns true when document is updated 10ms
✓ findManagerByEmail - returns Manager when email is found 10ms
✓ deleteManager - returns true when document is deleted 10ms

29 tests passed. 29 tests total. Time: 100ms

C:\Program Files\Java\jdk-21\bin\java.exe
WARNING: A Java agent has been loaded dynamically
WARNING: If a serviceability tool is in use, the
WARNING: Dynamic loading of agents will be disabled
Java HotSpot(TM) 64-Bit Server VM warning: Shared
Run 04, 2024 1:25:14 PM com.mongodb.internal.sasl
WARNING: SASL not found on the classpath. Using
MongoDB connection initialized successfully!
=== ManagerDAO.validateManager ===
Email: manager@hotel.com
Password hash: hashed_password
= Exception in validateManager: Connection lost
java.lang.RuntimeException: Connection lost
at dataAccessInject.ManagerDAO.validateManager
at com.ManagerDAOTest.validateManager_exception
    at org.junit.runner
=== ManagerDAO.validateManager ===
Email: manager@hotel.com
Password hash: hashed_password
Manager with email exists: true
Stored password hash: hashed_password
Hashes match: true
✓ Manager validation successful
java.lang.RuntimeException: Connection lost
Delete
```

Figure 19 ManagerDAOTest

5.3 Test Plan

Test Case ID	Test Description	Input	Expected Result
TC01	Guest login with valid credentials	Correct username & password	Login successful
TC02	Guest login with invalid password	Wrong password	Error message displayed
TC03	Guest login with empty fields	Blank username/password	Validation message
TC04	Guest registration with valid details	All required data	Account created
TC05	Guest registration with existing username	Duplicate username	Registration rejected
TC06	Receptionist login authentication	Valid staff login	Access granted
TC07	Manager login authentication	Valid manager login	Manager dashboard opens
TC08	Receptionist Add new reservation	Valid booking details	Reservation saved
TC09	Add reservation with missing data	Missing guest name	Validation error
TC10	Receptionist can edit guest account details	Updated Phone Number	Guest account details updated successfully
TC11	Calculate bill correctly	Check-in/out dates	Correct total amount
TC12	Bill printing	Reservation selected	Bill generated
TC13	Room details display	Request room list	Room info shown
TC14	Guest adds reservation	Valid booking info	Reservation stored
TC15	Room status automatically updates to "Occupied"	Change room status	Status updated
TC16	Receptionist marking room cleaning status	Cleaning update	Status saved
TC17	Delete guest record	Valid guest ID	Guest removed
TC18	Check-in process	Valid reservation	Guest checked in
TC19	Check-out process	Checked-in guest	Room released
TC20	Receptionist can calculate bill	Stay duration	Accurate bill
TC21	Manager adds a new room with valid details	Room No, Type, Floor, Price, Amenities	Room added successfully
TC22	Manager enters duplicate room number	Existing Room No	Error message displayed
TC23	Change room status to Available	Room status update	Status updated successfully
TC24	Manager creates receptionist account	Employee details	Receptionist account created
TC25	Manager deletes receptionist	Employee ID	Account removed

TC26	Manager can edit guest account details and receptionist account details	Modify receptionist contact number Modify guest phone number	Guest and receptionist account details updated successfully
TC27	Manager removes guest account	Guest ID	Guest account deleted
TC28	Manager calculates bill	Reservation details	Correct total bill
TC29	bill and sends it to the guest via email	Bill sent to registered guest email address	email sent successfully
TC30	Exit system	Exit option selected	Application closes safely

Figure 20 Test Plan

5.4 Test Cases

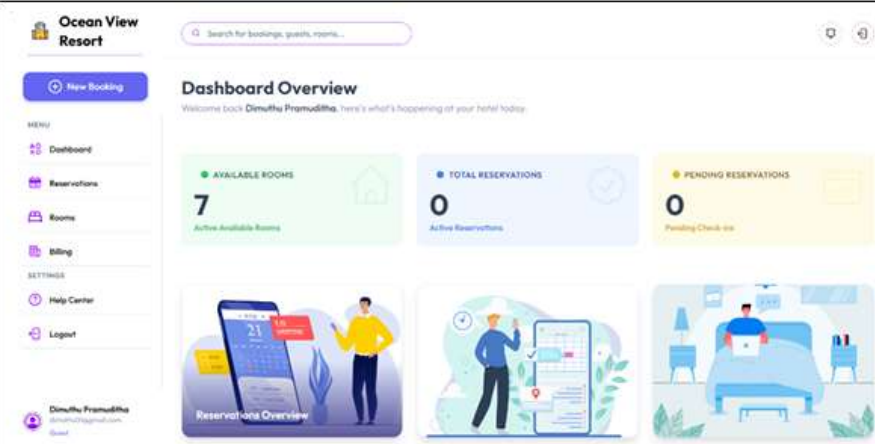
Test Case ID	TC01
Test Case Description	Verify guest login with valid credentials
Prerequisites	Guest account must already be registered in the system
Test Steps	1. Open login page 2. Enter valid username 3. Enter correct password 4. Click “Login” button
Test Data	Email: dimuthu01@gmail.com Password: dimuthu123
Expected Result	Login successful and user redirected to guest dashboard
Actual Result	Login successful and redirected to dashboard
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	

Figure 21 Test Case 01

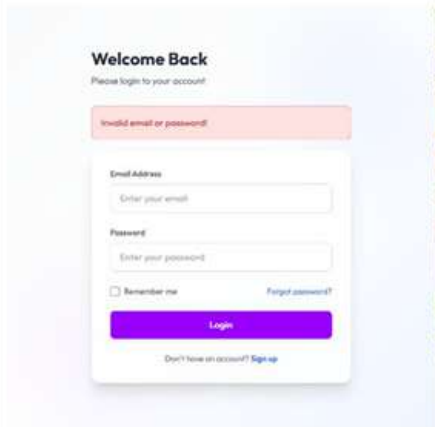
Test Case ID	TC02
Test Case Description	Verify guest login with invalid password
Prerequisites	Guest account must already exist
Test Steps	1. Open login page 2. Enter valid username 3. Enter incorrect password 4. Click "Login"
Test Data	Email: dimuthu01@gmail.com Password: Wrong123
Expected Result	System displays error message "Invalid password"
Actual Result	Error message displayed
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	 

Figure 22 Test Case 02

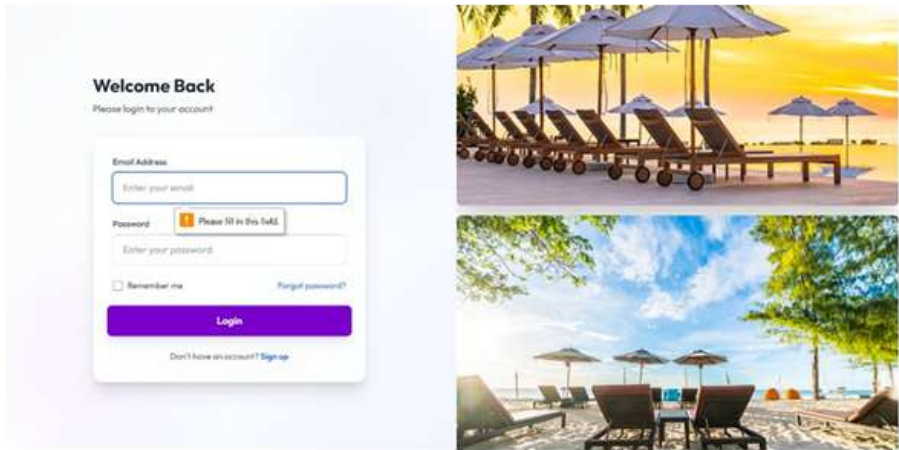
Test Case ID	TC03
Test Case Description	Verify guest login with empty fields
Prerequisites	Login page must be accessible
Test Steps	1. Open login page 2. Leave username blank 3. Leave password blank 4. Click "Login"
Test Data	Email: (Blank) Password: (Blank)
Expected Result	Validation message displayed "Fields cannot be empty"
Actual Result	Validation message displayed
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	

Figure 23 Test Case 03

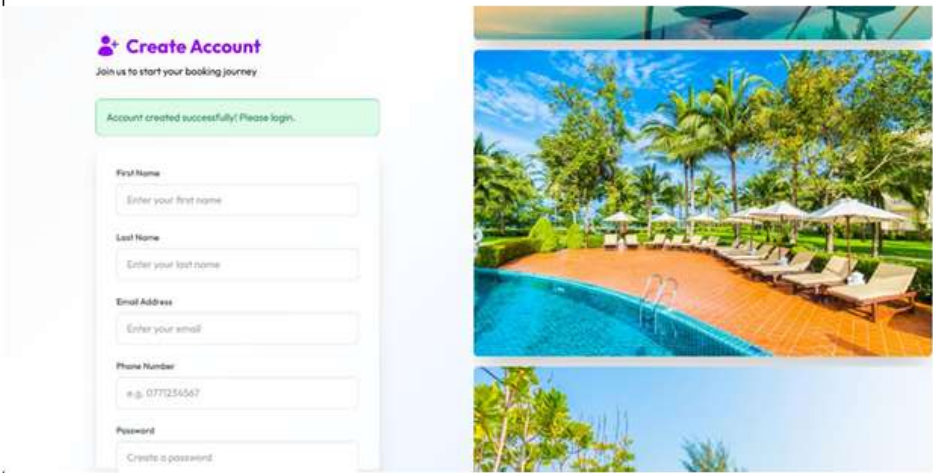
Test Case ID	TC04
Test Case Description	Verify guest registration with valid details
Prerequisites	Registration page must be accessible
Test Steps	1. Open registration page 2. Enter valid details in all required fields 3. Click “Register”
Test Data	First Name: Dimuthu Last Name: Pramuditha Email: dimuthu06@email.com Phone Number:0773658429 Password: Dimuthu@123
Expected Result	Account created successfully and confirmation message displayed
Actual Result	Account created successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	 The screenshot displays a 'Create Account' web form on the left and a resort pool area on the right. The form includes a title 'Create Account' with a user icon, a subtitle 'Join us to start your booking journey', and a green success message: 'Account created successfully! Please login...'. Below this are input fields for 'First Name' (placeholder: 'Enter your first name'), 'Last Name' (placeholder: 'Enter your last name'), 'Email Address' (placeholder: 'Enter your email'), 'Phone Number' (placeholder: 'e.g. 077254567'), and 'Password' (placeholder: 'Create a password'). The right side of the screenshot shows a vibrant outdoor pool with orange tiles, lounge chairs, and palm trees under a clear blue sky.

Figure 24 Test Case 04

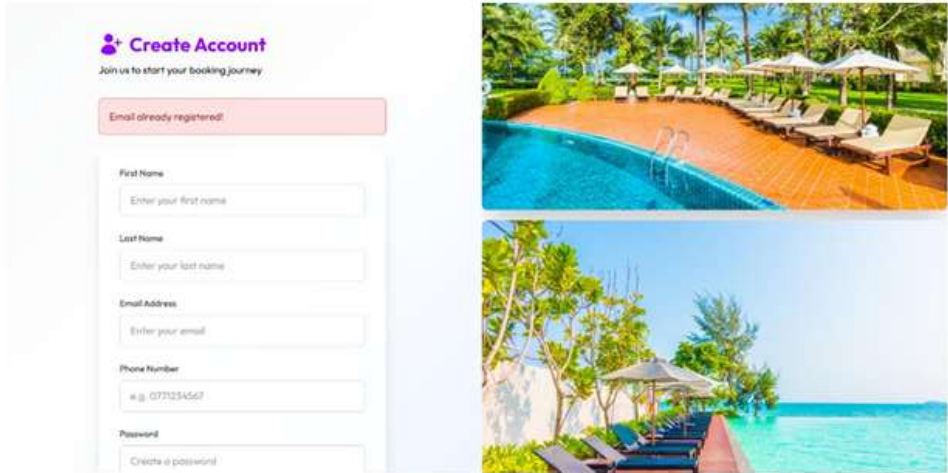
Test Case ID	TC05
Test Case Description	Verify guest registration with existing Email
Prerequisites	Username already exists in the system
Test Steps	1. Open registration page 2. Enter existing Email 3. Fill other required fields 4. Click “Register”
Test Data	First Name: Dimuthu Last Name: Pramuditha Email: dimuthu06@email.com Phone Number:0773658429 Password: Dimuthu@123
Expected Result	Registration rejected and error message displayed “Username already exists”
Actual Result	Registration rejected with error message
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	 The screenshot displays a 'Create Account' web form. At the top, it says 'Create Account' with a user icon and 'Join us to start your booking journey'. Below this, a red error message box states 'Email already registered!'. The form includes input fields for 'First Name', 'Last Name', 'Email Address', 'Phone Number' (with an example 'e.g. 0773234567'), and 'Password' (with a prompt 'Create a password'). To the right of the form are two images: the top one shows a resort pool with lounge chairs and umbrellas, and the bottom one shows a beach with lounge chairs and a building.

Figure 25 Test Case 05

Test Case ID	TC06
Test Case Description	Verify receptionist login authentication with valid staff credentials
Prerequisites	Receptionist account must be registered in the system
Test Steps	1. Open login page 2. Enter valid receptionist Email 3. Enter correct password 4. Click “Login”
Test Data	Email: receptionist@hotel.com Password: receptionist123
Expected Result	Access granted and receptionist dashboard displayed
Actual Result	Access granted and dashboard displayed
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	

Figure 26 Test Case 06

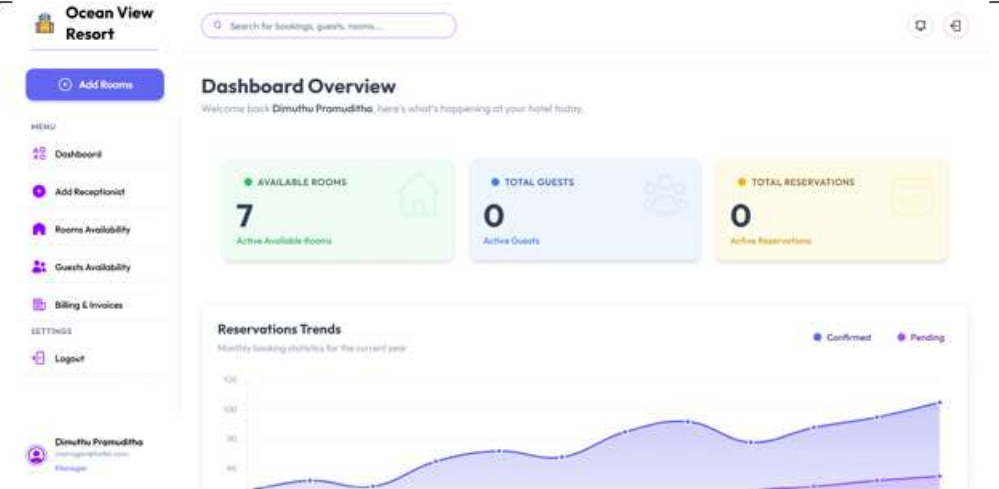
Test Case ID	TC07
Test Case Description	Verify manager login authentication with valid manager credentials
Prerequisites	Manager account must be registered in the system
Test Steps	1. Open login page 2. Enter valid manager Email 3. Enter correct password 4. Click “Login”
Test Data	Email: manager@hotel.com Password: manager123
Expected Result	Manager dashboard opens successfully
Actual Result	Manager dashboard displayed
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	

Figure 27 Test Case 07

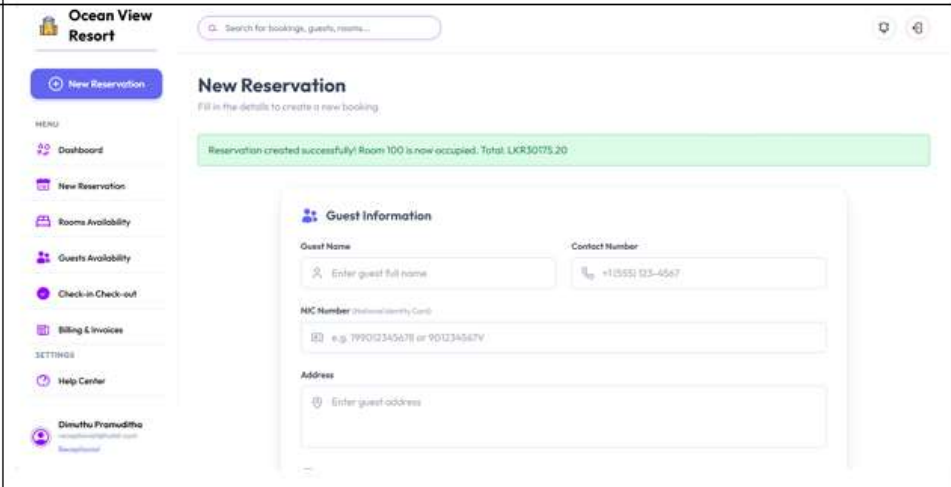
Test Case ID	TC08
Test Case Description	Verify adding a new reservation with valid booking details
Prerequisites	User must be logged in as receptionist
Test Steps	1. Navigate to “Add Reservation” page 2. Enter all required booking details 3. Click “Save” button
Test Data	Guest Name: Dimuthu Pramuditha Room No: 100 Check-in: 27/02/2026 Check-out: 01/03/2026
Expected Result	Reservation saved successfully and confirmation message displayed
Actual Result	Reservation saved successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	 The screenshot displays the 'New Reservation' page of the Ocean View Resort system. A green success message at the top states: 'Reservation created successfully! Room 100 is now occupied. Total: LKR30175.20'. Below this, the 'Guest Information' form is visible, containing fields for Guest Name, Contact Number, NIC Number, and Address. The left sidebar shows navigation options like Dashboard, New Reservation, Rooms Availability, and a user profile for Dimuthu Pramuditha.

Figure 28 Test Case 08

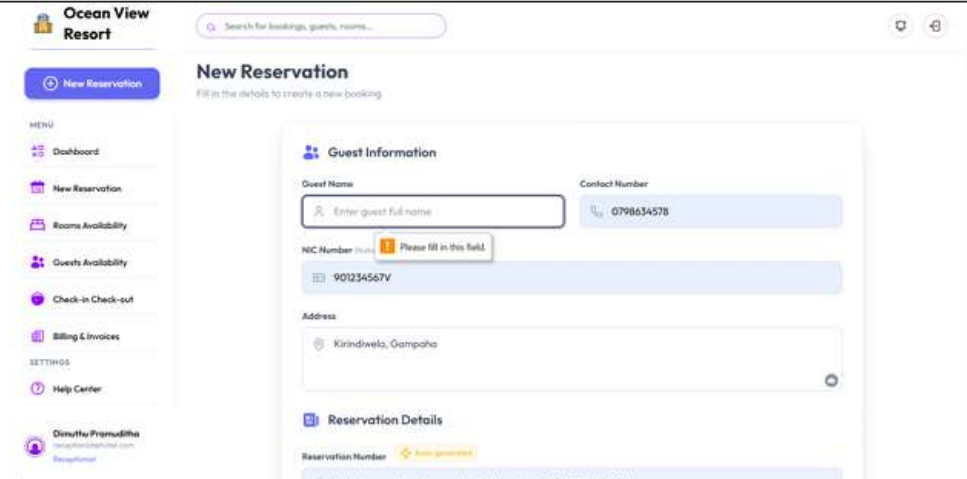
Test Case ID	TC09
Test Case Description	Verify adding reservation with missing required data
Prerequisites	User must be logged in
Test Steps	1. Navigate to “Add Reservation” page 2. Leave Guest Name field empty 3. Enter other required details 4. Click “Save”
Test Data	Guest Name: (Blank) Room No: 102 Check-in: 01/03/2026 Check-out: 05/03/2026
Expected Result	Validation error displayed “Guest name is required”
Actual Result	Validation message displayed
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	26/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	26/02/2026
Screenshot	

Figure 29 Test Case 09

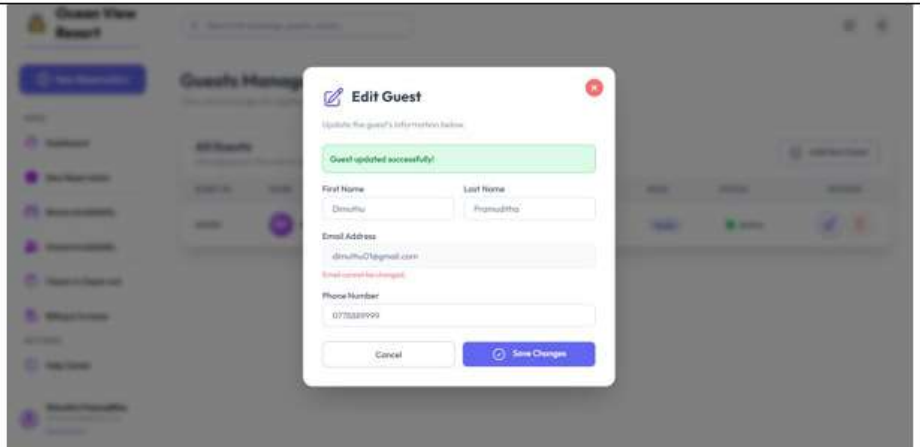
Test Case ID	TC10
Test Case Description	Verify that receptionist can edit guest account details successfully
Prerequisites	1. Guest account must already exist in the system 2. Receptionist must be logged into the system
Test Steps	1. Login as receptionist 2. Navigate to “Manage Guests” section 3. Search using valid Guest ID 4. Click “Edit” button 5. Modify guest details (e.g., phone number or email) 6. Click “Update”
Test Data	Guest ID: #G001 Updated Phone Number: 0778889999
Expected Result	1. Guest account details updated successfully 2. Updated information saved in the system database 3. Confirmation message displayed
Actual Result	Guest account details updated successfully and saved
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026
Screenshot	 The screenshot displays a web application interface with a modal window titled 'Edit Guest'. The modal contains a green success message: 'Guest updated successfully!'. Below this, there are input fields for 'First Name' (containing 'Dimuthu'), 'Last Name' (containing 'Pramuditha'), 'Email Address' (containing 'dimuthu01@gmail.com'), and 'Phone Number' (containing '0778889999'). At the bottom of the modal are two buttons: 'Cancel' and 'Save Changes' (which is highlighted in blue). The background shows a blurred view of the 'Guests Management' section of the application.

Figure 30 Test Case 10

Test Case ID	TC11
Test Case Description	Verify system calculates bill correctly based on check-in and check-out dates
Prerequisites	Reservation must exist with valid check-in and check-out dates
Test Steps	1. Login as receptionist 2. Select Invoice existing reservation 3. Click “Generate Bill” 4. Verify total amount calculation
Test Data	Room Rate: LKR 12700.00 per night Check-in: 27/02/2026 Check-out: 01/03/2026 (2 nights)
Expected Result	Sub Total amount calculated correctly (LKR 25400.00) with Service (10%) and Tax (8%) Total Amount (LKR 30175.20)
Actual Result	Total amount calculated as LKR 30175.20
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026

Screenshot

**Ocean View Resort**

Search for bookings, guests, rooms...

New Reservation

Dashboard

New Reservation

Rooms Availability

Guests Availability

Check-in Check-out

Billing & Invoices

SETTINGS

**Dimuthu Pramuditha**
reservations@oceanviewresort.com
Reservations

Billing & Invoices
Generate and manage reservation invoices

Select Invoice: Invoice #RES-2026-001 - Dimuthu Pramuditha 

INVOICE

Ocean View Resort
Luxury Hotel & Spa


Invoice #RES-2026-001
Date: Feb. 27, 2026

Guest Details

Guest Name

Dimuthu Pramuditha

Contact Number

0798634578

Address

Kirindiwela, Gampaha

Booking Details

Reservation Number

RES-2026-001

Room Number

100

Room Type

Double

Check-in Date

Feb 27, 2026

Check-out Date

Mar 01, 2026

Number of Nights

2 Nights

Bill Summary

Description	Quantity	Rate	Amount
Room Charges	2 nights	LKR 12700.00	LKR 25400.00
Subtotal (Room Charges):			LKR 25400.00
Service (10%):			LKR 2540.00
Tax (8%):			LKR 2235.20
TOTAL AMOUNT:			LKR 30175.20

Thank You for Your Stay!

We hope you enjoyed your experience at Ocean View Resort.
We look forward to welcoming you again soon!

Contact Information

Phone
+1(555) 123-4567

Email
info@oceanviewresort.com

Address
123 Ocean Drive, Beach City

This is a computer-generated invoice and does not require a signature.
For any queries, please contact our reception desk.

Figure 31 Test Case 11

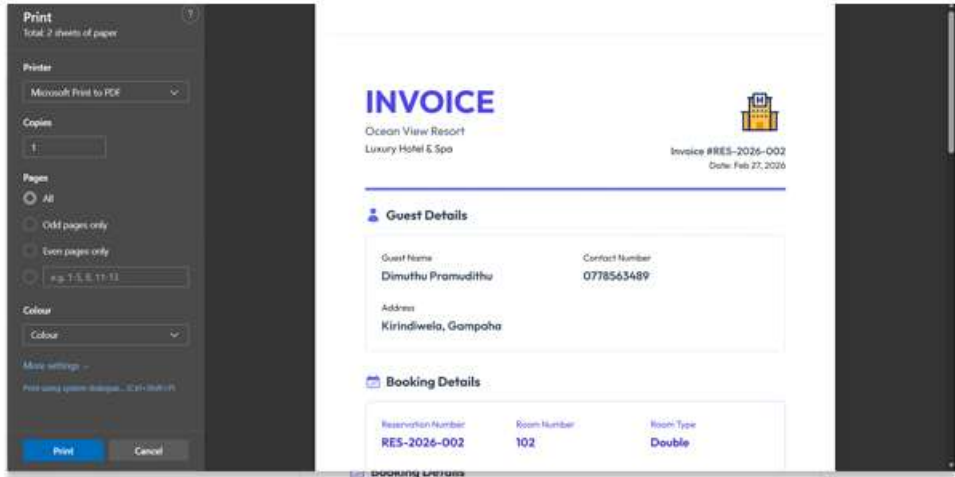
Test Case ID	TC12
Test Case Description	Verify bill printing functionality
Prerequisites	Reservation must be selected and bill must be generated
Test Steps	1. Login to system 2. Select existing reservation 3. Click “Print Bill” button
Test Data	Reservation No: RES-2026-002
Expected Result	Bill generated and printable invoice displayed
Actual Result	Bill generated successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026
Screenshot	

Figure 32 Test Case 12

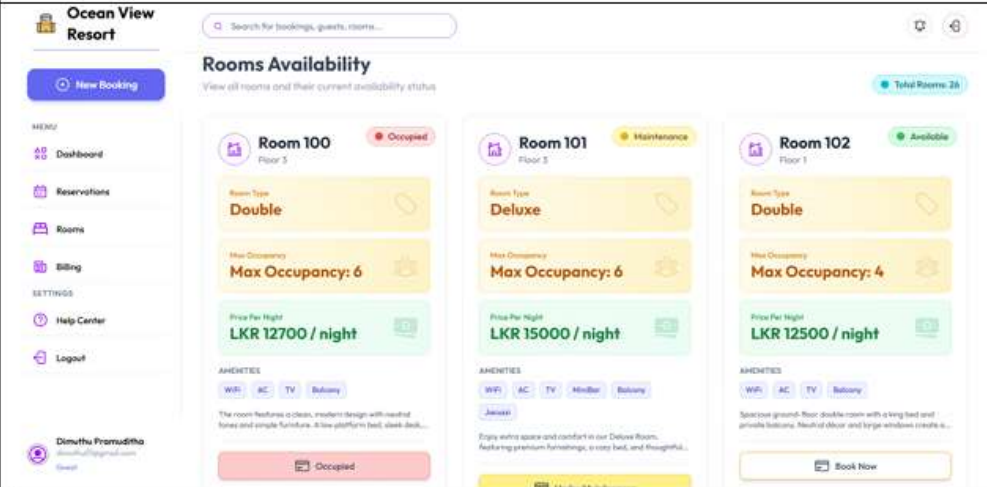
Test Case ID	TC13
Test Case Description	Verify room details display functionality
Prerequisites	Rooms must be available in the system database
Test Steps	1. Login to system 2. Navigate to “Rooms” section 3. Click “Display Room List”
Test Data	Request: All available rooms
Expected Result	Room information (Room No, Type, Price, Status) displayed correctly
Actual Result	Room information displayed correctly
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026
Screenshot	 The screenshot displays the 'Rooms Availability' section of the Ocean View Resort management interface. It features a sidebar menu on the left with options like Dashboard, Reservations, Rooms, Billing, Help Center, and Logout. The main content area shows three room cards. Room 100 (Floor 3) is a Double room, currently Occupied, with a max occupancy of 6 and a price of LKR 12700 per night. Room 101 (Floor 3) is a Deluxe room, currently under Maintenance, with a max occupancy of 6 and a price of LKR 15000 per night. Room 102 (Floor 1) is a Double room, currently Available, with a max occupancy of 4 and a price of LKR 12500 per night. Each card also lists amenities such as WiFi, AC, TV, Balcony, and a minibar.

Figure 33 Test Case 13

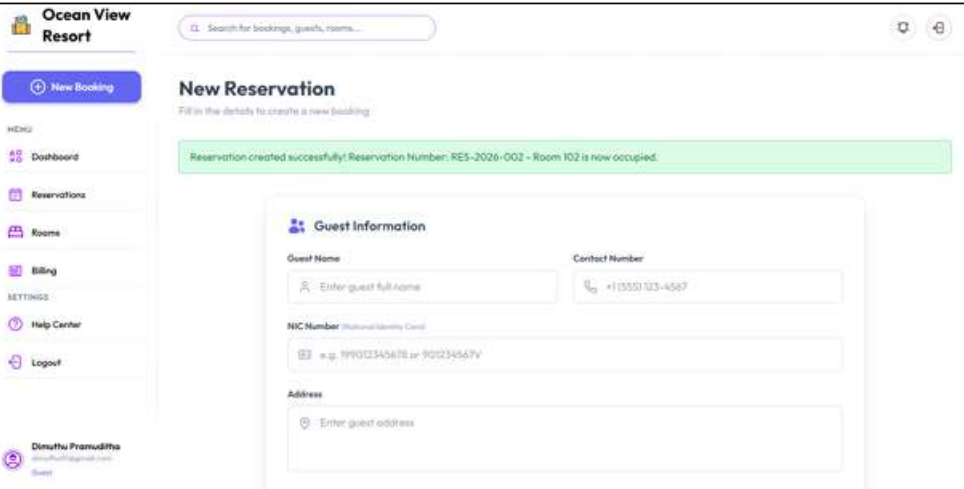
Test Case ID	TC14
Test Case Description	Verify guest adds reservation with valid booking information
Prerequisites	Guest must be logged into the system
Test Steps	1. Login as guest 2. Navigate to “Add Reservation” 3. Enter valid booking details 4. Click “Submit”
Test Data	Guest Name: Dimuthu Pramuditha Reservation No: RES-2026-002 Room Type: Double Check-in: 28/02/2026 Check-out: 03/03/2026
Expected Result	Reservation stored successfully and confirmation message displayed
Actual Result	Reservation stored successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026
Screenshot	 The screenshot displays the 'New Reservation' form in the Ocean View Resort system. A green success message at the top states: 'Reservation created successfully! Reservation Number: RES-2026-002 - Room 102 is now occupied.' The form includes fields for Guest Name (filled with 'Dimuthu Pramuditha'), Contact Number (filled with '+1 (355) 323-4567'), NIC Number (filled with '990123456789'), and Address (filled with '123 Main Street, Colombo, Sri Lanka'). The user is logged in as Dimuthu Pramuditha.

Figure 34 Test Case 14

Test Case ID	TC15
Test Case Description	Verify that room status automatically updates to “Occupied” when a reservation is created using that room number
Prerequisites	1. Room No 102 must exist in the system 2. Room status must be “Available” before test execution
Test Steps	1. Login as Guest 2. Navigate to “Add Reservation” 3. Enter valid guest details 4. Select Room No 102 5. Enter check-in and check-out dates 6. Click “Save” 7. Navigate to “Rooms” and check Room No 102 status
Test Data	Guest Name: Dimuthu Pramuditha Room No: 102 Check-in: 28/02/2026 Check-out: 03/03/2026
Expected Result	1. Reservation saved successfully 2. Room No 102 status automatically changes to “Occupied”
Actual Result	Reservation saved successfully and room status updated to “Occupied”
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026
Screenshot	

Figure 35 Test Case 15

Test Case ID	TC16
Test Case Description	Verify marking room cleaning status and saving the updated status
Prerequisites	1. Room must exist in the system
Test Steps	1. Login as Receptionist 2. Navigate to “Rooms Availability” 3. Select Room No 204 4. Change status to “Cleaning” 5. Click “Update”
Test Data	Room No: 204 New Status: Cleaning
Expected Result	Room cleaning status saved successfully in the system
Actual Result	Room status updated to “Cleaning” successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	The screenshot displays the 'Rooms Availability' section of the Ocean View Resort management system. A modal window is open, showing a confirmation message: 'Room 204 status updated to Cleaning'. The interface lists several rooms with their respective details and status buttons. Room 204 is highlighted with a yellow 'Housekeeping' tag. Other rooms shown include Room 202, Room 205, Room 206, and Room 310, each with options to set status to 'Available', 'House', or 'Clean'.

Figure 36 Test Case 16

Test Case ID	TC17
Test Case Description	Verify deleting a guest record using a valid guest ID
Prerequisites	Guest record must already exist in the system
Test Steps	1. Login as Receptionist 2. Navigate to “Guests Management” 3. Valid Guest ID 4. Click “Delete” 5. Confirm deletion
Test Data	Guest ID: #G002
Expected Result	Guest record removed successfully from the system
Actual Result	Guest record deleted successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	

Figure 37 Test Case 17

Test Case ID	TC18
Test Case Description	Verify check-in process for a valid reservation
Prerequisites	1. Reservation must exist
Test Steps	1. Login as receptionist 2. Navigate to “Check-in Check-out” 3. Find valid reservation number 4. Click “Check-in”
Test Data	Reservation No: RES-2026-002
Expected Result	1. Guest check-in successful 2. Room status updated to “Occupied”
Actual Result	Guest checked in successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	

Figure 38 Test Case 18

Test Case ID	TC19
Test Case Description	Verify check-out process and automatic room status update to “Cleaning”
Prerequisites	1. Guest must be already checked-in 2. Room status must be “Occupied”
Test Steps	1. Login as receptionist 2. Navigate to “Check-in Check-out” 3. Select guest record 4. Click “Check-out” 5. Confirm action
Test Data	Reservation No: RES-2026-003 Room No:103
Expected Result	1. Guest check-out successful 2. Room status automatically updated to “Cleaning”
Actual Result	Guest checked out successfully and room status updated to “Cleaning”
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	The screenshot shows the 'Ocean View Resort' management interface. A notification at the top indicates 'Reservation status updated to Checked Out'. The main area is divided into two sections: 'Check-in Check-out' and a room status grid. The 'Check-in Check-out' section shows three guest records for 'Double' rooms, all marked 'Checked In'. The room status grid shows six rooms: Room 103 (Cleaning), Room 104 (Available), Room 201 (Available), Room 202 (Available), Room 203 (Cleaning), and Room 204 (Cleaning). Each room card displays its type, price per night, and current status.

Figure 39 Test Case 19

Test Case ID	TC20
Test Case Description	Verify receptionist can calculate bill accurately based on stay duration
Prerequisites	1. Guest must be checked-in 2. Room rate must be defined in system
Test Steps	1. Login as receptionist 2. Select Invoice checked-in guest 3. Click "Calculate Bill" 4. Verify total amount
Test Data	Room Rate: LKR 12500 per night Stay Duration: 3 nights
Expected Result	Accurate bill calculated (LKR 44550) add Service (10%) and Tax (8%) based on stay duration
Actual Result	Bill calculated correctly as LKR 44550
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026

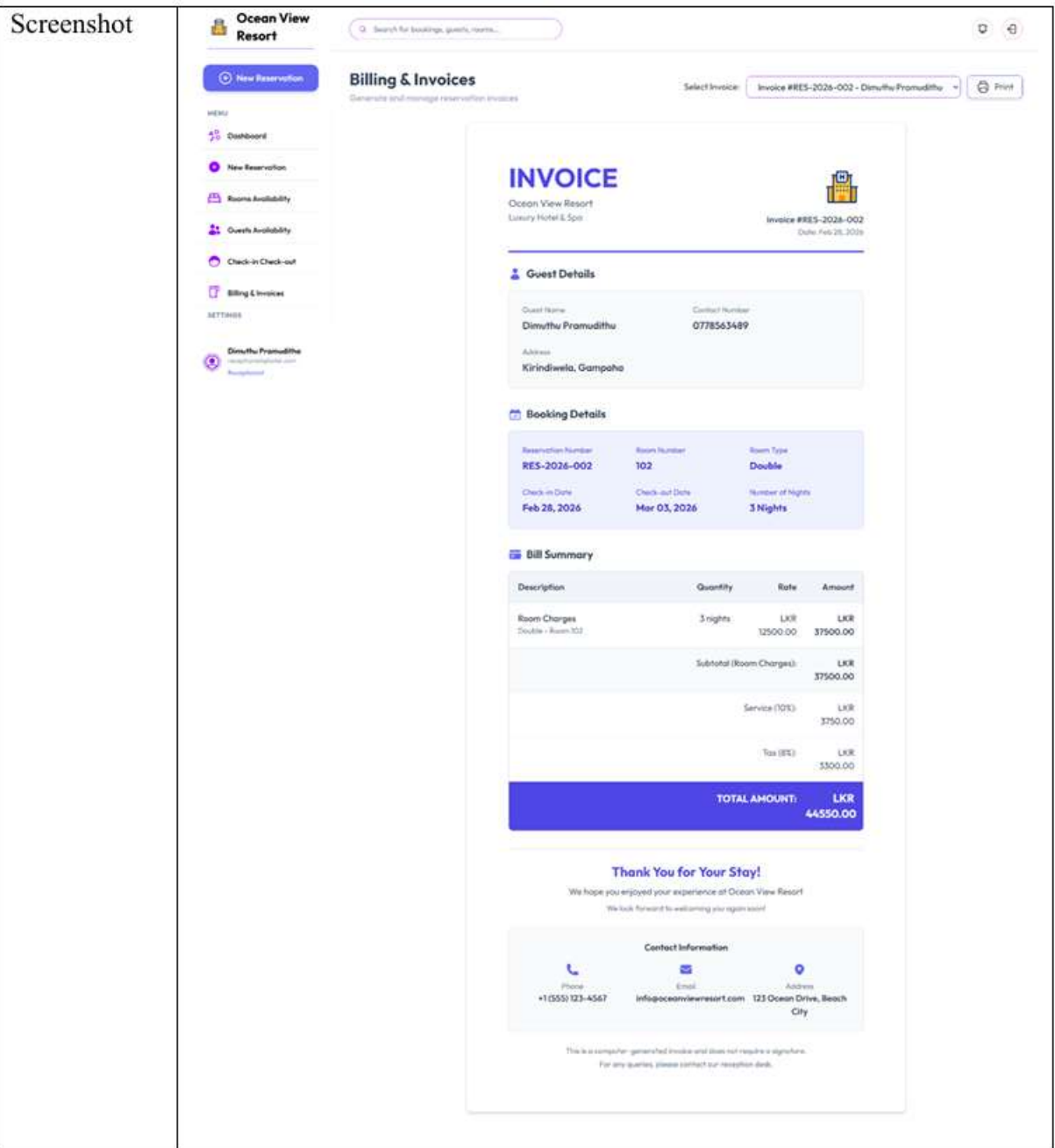


Figure 40 Test Case 20

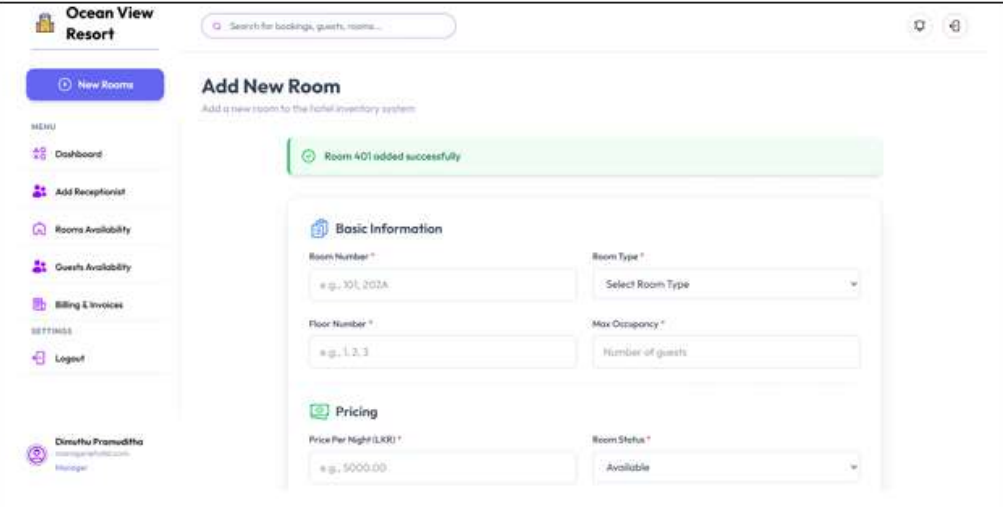
Test Case ID	TC21
Test Case Description	Verify manager can add a new room with valid details
Prerequisites	Manager must be logged into the system
Test Steps	1. Login as manager 2. Navigate to “Manage Rooms” 3. Click “Add New Room” 4. Enter valid room details 5. Click “Save”
Test Data	Room No: 401 Type: Family Floor: 4 Price: LKR 42,000 Amenities: WiFi, AC, TV, Minibar, Balcony, Work Desk
Expected Result	Room added successfully and appears in room list
Actual Result	Room added successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	 The screenshot displays the 'Ocean View Resort' management interface. On the left is a sidebar menu with options like Dashboard, Add Receptionist, Rooms Availability, Guests Availability, Billing & Invoices, and Logout. The main area is titled 'Add New Room' and contains a form with the following fields: Room Number (with example '101, 202A'), Room Type (dropdown menu), Floor Number (with example '1, 2, 3'), Max Occupancy (Number of guests), Price Per Night (LKR) (with example '5000.00'), and Room Status (dropdown menu with 'Available' selected). A green success message at the top of the form area reads 'Room 401 added successfully'.

Figure 41 Test Case 21

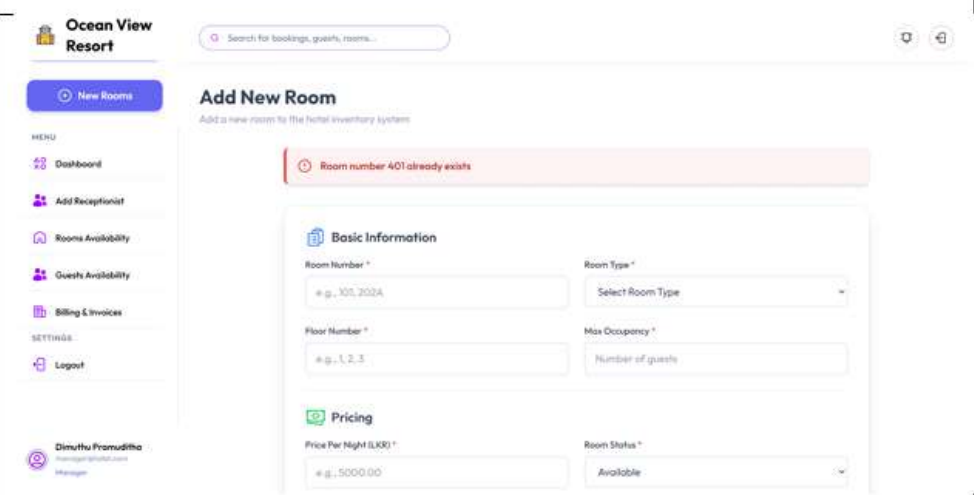
Test Case ID	TC22
Test Case Description	Verify system prevents adding duplicate room number
Prerequisites	Room No 401 must already exist in the system
Test Steps	1. Login as manager 2. Navigate to “Add New Room” 3. Enter existing Room No 401 4. Fill other details 5. Click “Save”
Test Data	Room No: 401 (Existing)
Expected Result	Error message displayed “Room number already exists” and room not added
Actual Result	Error message displayed and room not added
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	

Figure 42 Test Case 22

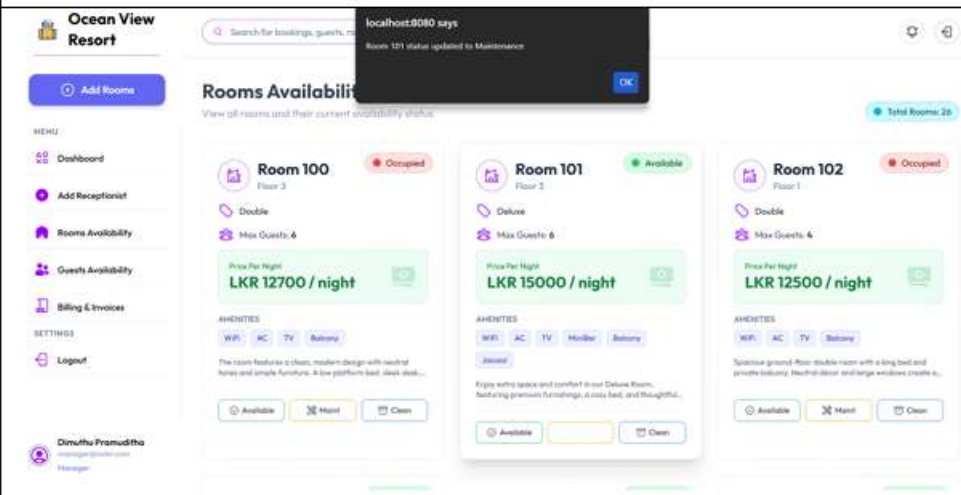
Test Case ID	TC23
Test Case Description	Verify updating room availability status
Prerequisites	Room must already exist in the system
Test Steps	1. Login as manager 2. Navigate to “Rooms Availability” 3. Select room 4. Change status (Available/ Cleaning /Maintenance) 5. Click “Update”
Test Data	Room No: 101 Status: Maintenance
Expected Result	Room status updated successfully in system
Actual Result	Room status updated successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	 The screenshot shows the 'Ocean View Resort' management dashboard. A central modal window titled 'Rooms Availability' displays a success message: 'Room 101 status updated to Maintenance'. Below the modal, three room cards are visible: Room 100 (Floor 3, Double, Occupied, LKR 12700/night), Room 101 (Floor 3, Deluxe, Available, LKR 15000/night), and Room 102 (Floor 1, Double, Occupied, LKR 12500/night). Each card includes a list of amenities (WIFI, AC, TV, Balcony) and buttons for 'Available', 'Maintenance', and 'Clean'. The left sidebar contains navigation links for Dashboard, Add Receptionist, Rooms Availability, Guests Availability, Billing & Invoices, and Settings. The user 'Dimuthu Pramuditha' is logged in as a Manager.

Figure 43 Test Case 23

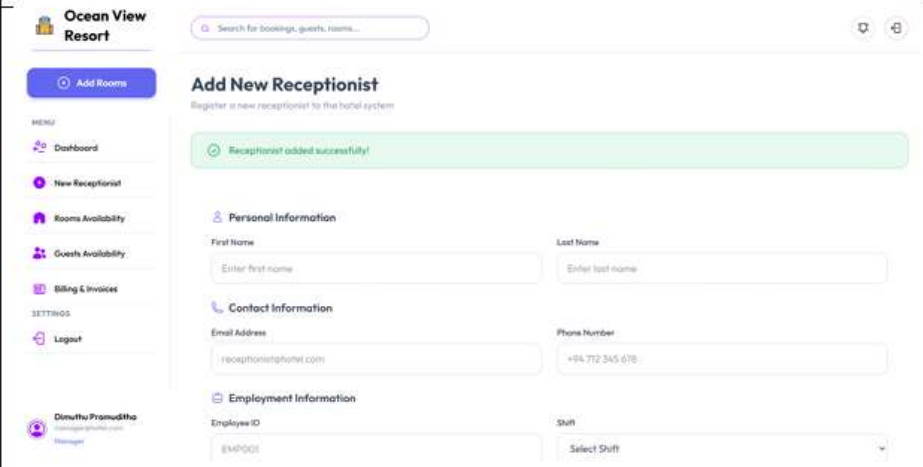
Test Case ID	TC24
Test Case Description	Verify manager can create a receptionist account with valid employee details
Prerequisites	Manager must be logged into the system
Test Steps	1. Login as manager 2. Navigate to “Manage dashboard” 3. Click “Add Receptionist” 4. Enter valid employee details 5. Click “Create Account”
Test Data	Name: Dimuthu Pramuditha Email: receptionist01@hotel.com Password: receptionist123 Contact: 0715478317
Expected Result	Receptionist account created successfully and stored in system
Actual Result	Receptionist account created successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	

Figure 44 Test Case 24

Test Case ID	TC25
Test Case Description	Verify manager can delete a receptionist account using valid Employee ID
Prerequisites	1. Receptionist account must already exist in the system 2. Manager must be logged in
Test Steps	1. Login as manager 2. Navigate to “Guests & Staff Management” 3. Search using valid Employee ID 4. Click “Delete” 5. Confirm deletion
Test Data	Employee ID: EMP002
Expected Result	Receptionist account removed successfully from the system
Actual Result	Receptionist account deleted successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	

Figure 45 Test Case 25

Test Case ID	TC26
Test Case Description	Verify manager can edit guest account details and receptionist account details successfully
Prerequisites	1. Guest account must already exist in the system 2. Receptionist account must already exist 3. Manager must be logged into the system
Test Steps (Guest Account Edit)	1. Login as manager 2. Navigate to “Guests & Staff Management” 3. Find valid Guest ID 4. Click “Edit” 5. Modify guest details (e.g., phone number) 6. Click “Update”
Test Steps (Receptionist Account Edit)	1. Navigate to “Guests & Staff Management” 2. Find valid Employee ID 3. Click “Edit” 4. Modify receptionist details (e.g., contact number) 5. Click “Update”
Test Data	Guest ID: G1020 New Phone: 0775556677 Employee ID: EMP002 New Contact: 0711234567
Expected Result	1. Guest account details updated successfully 2. Receptionist account details updated successfully 3. Updated information saved in the system database
Actual Result	Guest and receptionist account details updated successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026

Screenshot

The image displays two screenshots of a mobile application interface, specifically the 'Guests & Staff' section. The top screenshot shows the 'Edit Receptionist' modal form, which is used to update a receptionist's information. The bottom screenshot shows the 'Edit Guest' modal form, used to update a guest's information. Both forms include fields for First Name, Last Name, Email Address, Phone Number, Shift, and Employee ID, along with 'Cancel' and 'Save Changes' buttons.

Top Screenshot: Edit Receptionist

Update the receptionist's information below.

Receptionist updated successfully!

First Name: DimaRus Last Name: PrasadRus

Email Address: receptionist@prasadRus.com

Phone Number: 071254567 Shift: Evening

Employee ID: EMP002

Buttons: Cancel, Save Changes

Bottom Screenshot: Edit Guest

Update the guest's information below.

Guest updated successfully!

First Name: DimaRus Last Name: PrasadRus

Email Address: dimaRus@prasadRus.com

Phone Number: 071254567

Buttons: Cancel, Save Changes

Figure 46 Test Case 26

Test Case ID	TC27
Test Case Description	Verify manager can remove a guest account using valid Guest ID
Prerequisites	1. Guest account must exist in the system 2. Manager must be logged in
Test Steps	1. Login as manager 2. Navigate to “Guests & Staff Management” 3. Search using valid Guest ID 4. Click “Delete” 5. Confirm action
Test Data	Guest ID: U002
Expected Result	Guest account deleted successfully from the system
Actual Result	Guest account deleted successfully
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	2821/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026

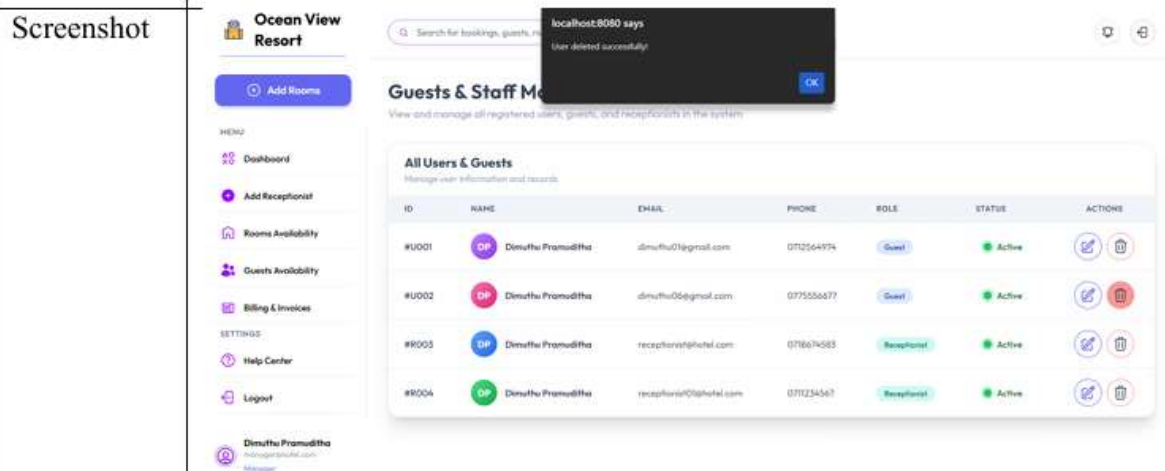


Figure 47 Test Case 27

Test Case ID	TC28
Test Case Description	Verify manager can calculate bill accurately using reservation details
Prerequisites	1. Reservation must exist 2. Room rate must be defined in the system
Test Steps	1. Login as manager 2. Navigate to “Billing & Invoices” 3. Select valid reservation 4. Click “Calculate Bill”
Test Data	Reservation No: RES-2026-002 Room Rate: LKR 12500 per night Stay Duration: 3 nights
Expected Result	Correct total bill calculated (LKR 44550.00) including Service (10%) and Tax (8%)
Actual Result	Total bill calculated correctly as LKR 44550.00
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026

Screenshot

**Ocean View Resort**

Search for bookings, guests, rooms...

Add Rooms

Dashboard

Add Receptionist

Rooms Availability

Guests Availability

Billing & Invoices

Settings

Help Center

Dimuthu Pramudithu

Logout

Billing & Invoices
Generate and manage reservation invoices

Select Invoice: Invoice #RES-2026-002 - Dimuthu Pramudithu 

INVOICE
Ocean View Resort
Luxury Hotel & Spa

**Invoice #RES-2026-002**
Date: Feb 28, 2026

Guest Details

Guest Name

Dimuthu Pramudithu

Contact Number

0778563489

Address

Kirindiwela, Gampaha

Booking Details

Reservation Number

RES-2026-002

Room Number

102

Room Type

Double

Check-in Date

Feb 28, 2026

Check-out Date

Mar 03, 2026

Number of Nights

3 Nights

Bill Summary

Description	Quantity	Rate	Amount
Room Charges	3 nights	LKR 12500.00	LKR 37500.00
Subtotal (Room Charges):			LKR 37500.00
Service (10%):			LKR 3750.00
Tax (10%):			LKR 3300.00
TOTAL AMOUNT:			LKR 44550.00

Thank You for Your Stay!

We hope you enjoyed your experience at Ocean View Resort.
We look forward to welcoming you again soon!

Contact Information

Phone

+1 (555) 123-4567

Email

info@oceanviewresort.com

Address

123 Ocean Drive, Beach City

This is a computer-generated invoice and does not require a signature.
For any queries, please contact our reception desk.

Figure 48 Test Case 28

Test Case ID	TC29
Test Case Description	Verify that when receptionist clicks the “Check-out” button, the system generates the bill and sends it to the guest via email
Prerequisites	1. Guest must be checked-in 2. Guest email address must be registered in the system 3. Internet/email service must be active 4. Room status must be “Occupied”
Test Steps	1. Login as receptionist 2. Navigate to “Checked-in Guests” 3. Select valid reservation 4. Click “Check-out” button 5. Confirm check-out action
Test Data	Reservation No: RES3030 Guest Email: guest3030@email.com Room Rate: LKR 10,000 per night Stay Duration: 2 nights
Expected Result	1. Guest successfully checked out 2. Final bill generated correctly 3. Bill sent to registered guest email address 4. Room status updated to “Cleaning”
Actual Result	Guest checked out successfully, bill generated, email sent successfully, and room status updated to “Cleaning”
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	27/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	27/02/2026

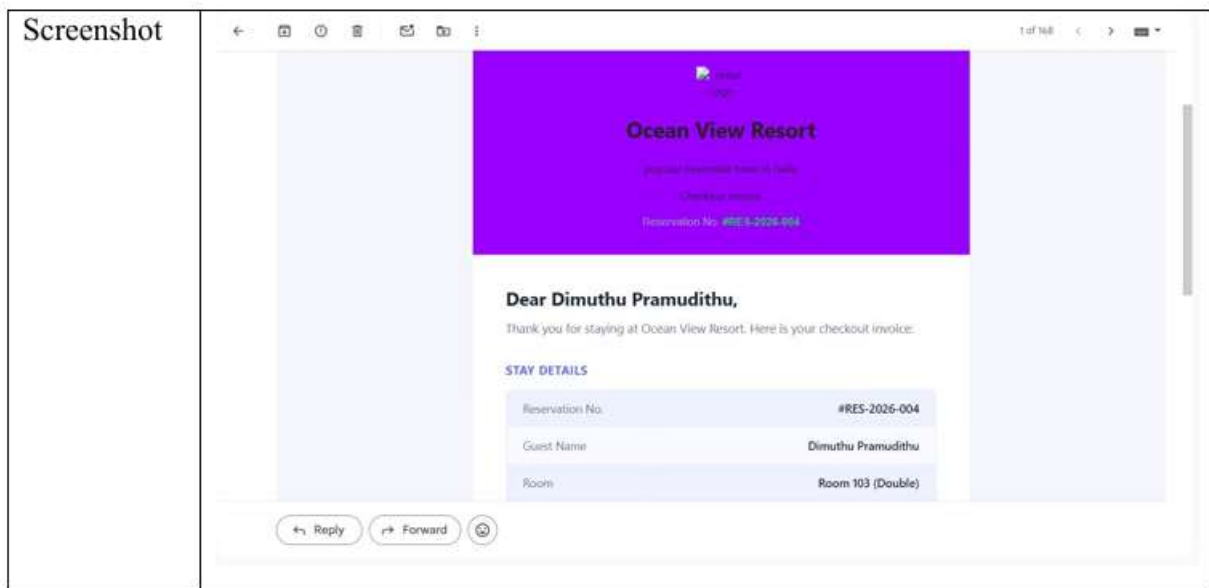


Figure 49 Test Case 29

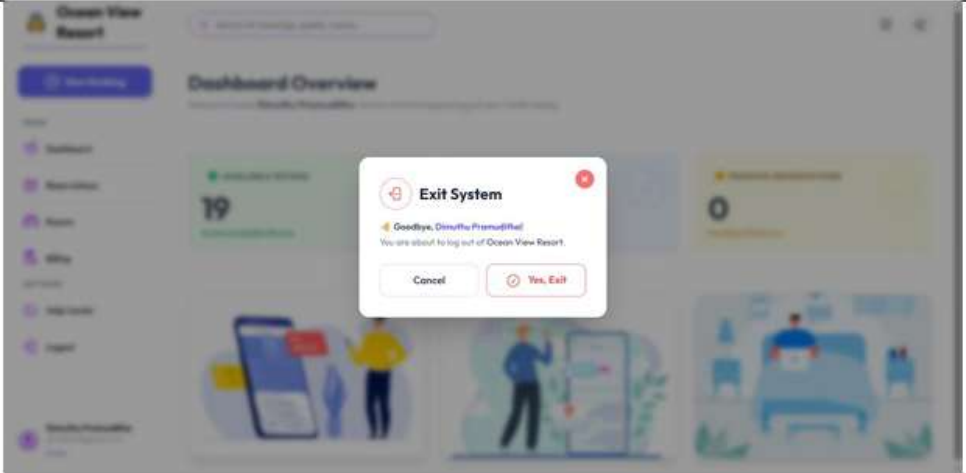
Test Case ID	TC30
Test Case Description	Verify that the system exits safely when the Exit option is selected
Prerequisites	1. User must be logged into the system 2. Application must be running
Test Steps	1. Login as any valid user (Manager/Receptionist/Guest) 2. Navigate to main dashboard 3. Click “Exit” option/button 4. Confirm exit if confirmation dialog appears
Test Data	Action: Select Exit option
Expected Result	1. Application closes safely without errors
Actual Result	Application closed successfully without errors
Status	Pass
Created By	Dimuthu Pramuditha
Date of Creation	28/02/2026
Executed By	Dimuthu Pramuditha
Date of Execution	28/02/2026
Screenshot	

Figure 50 Test Case 30

5.5 Requirement Traceability Matrix (RTM)

The Requirement Traceability Matrix ensures that all system requirements are properly tested. It maps each functional requirement to its corresponding test case to confirm complete test coverage.

Requirement ID	Requirement Description	Test Case ID(s)	Status
FR01	Guest registration	TC04, TC05	Covered
FR02	Guest login authentication	TC01, TC02, TC03	Covered
FR03	Staff login authentication	TC06, TC07	Covered
FR04	Add reservation	TC08, TC14	Covered
FR05	Reservation validation	TC09	Covered
FR06	Edit guest account	TC10, TC26	Covered
FR07	Bill calculation	TC11, TC20, TC28	Covered
FR08	Bill generation and printing	TC12	Covered
FR09	Display room details	TC13	Covered
FR10	Automatic room status updates	TC15, TC18, TC19, TC23	Covered
FR11	Cleaning status update	TC16	Covered
FR12	Check-in and check-out	TC18, TC19	Covered
FR13	Manage rooms	TC21, TC23	Covered
FR14	Prevent duplicate room numbers	TC22	Covered
FR15	Manage receptionist accounts	TC24, TC25, TC26	Covered
FR16	Manage guest accounts	TC17, TC27	Covered
FR17	Send bill via email	TC29	Covered
FR18	Safe system exit	TC30	Covered

Figure 51 Requirement Traceability Matrix (RTM)

5.6 Testing HTTP API Endpoints Using Postman

To validate the backend functionality of the Room Reservation System, HTTP API endpoints were tested using Postman. Postman allows sending HTTP requests and analyzing responses without interacting through the JSP user interface. This ensures that the Servlet controllers, business logic, and MongoDB database interactions function correctly in isolation.

5.6.1 Login API Testing

The login API authenticates users (Guest, Receptionist, or Manager) by validating their username and password against records stored in MongoDB.

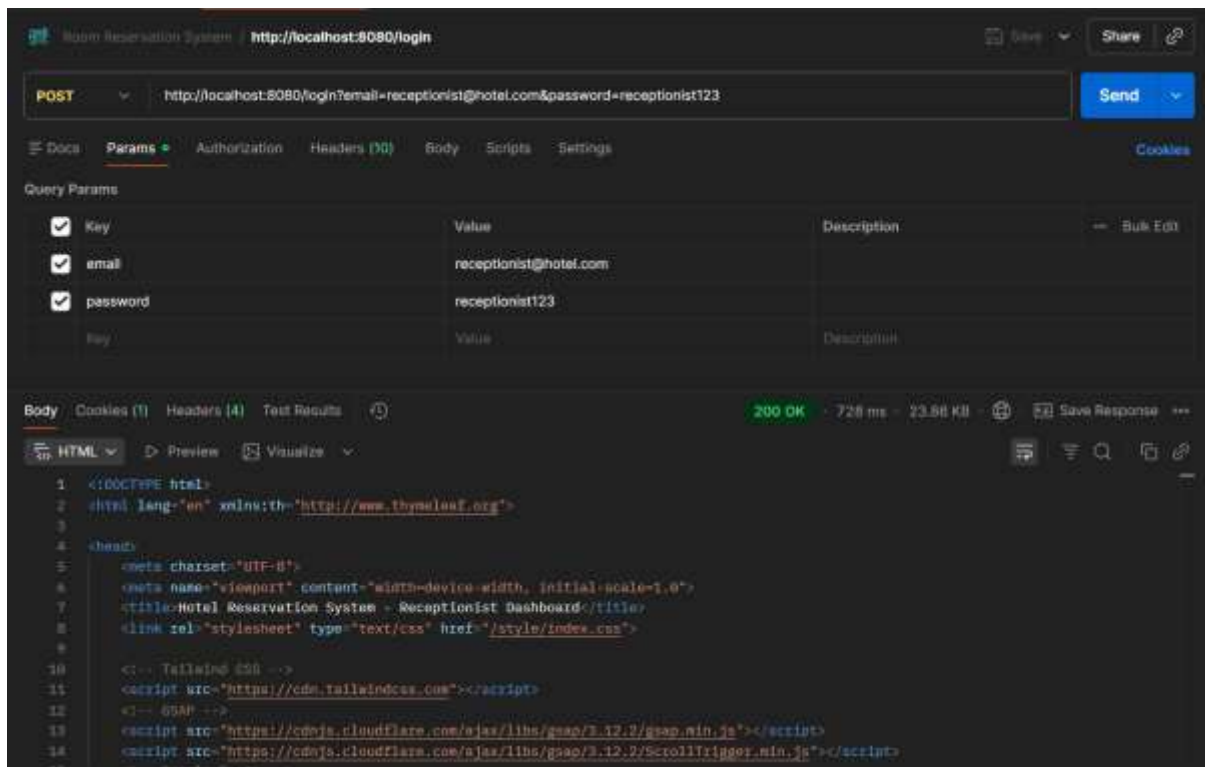


Figure 52 Login API Testing

5.6.2 Register API Testing

The register API allows new guests to create an account in the system. It validates user details and ensures duplicate usernames are not allowed before storing data in MongoDB.

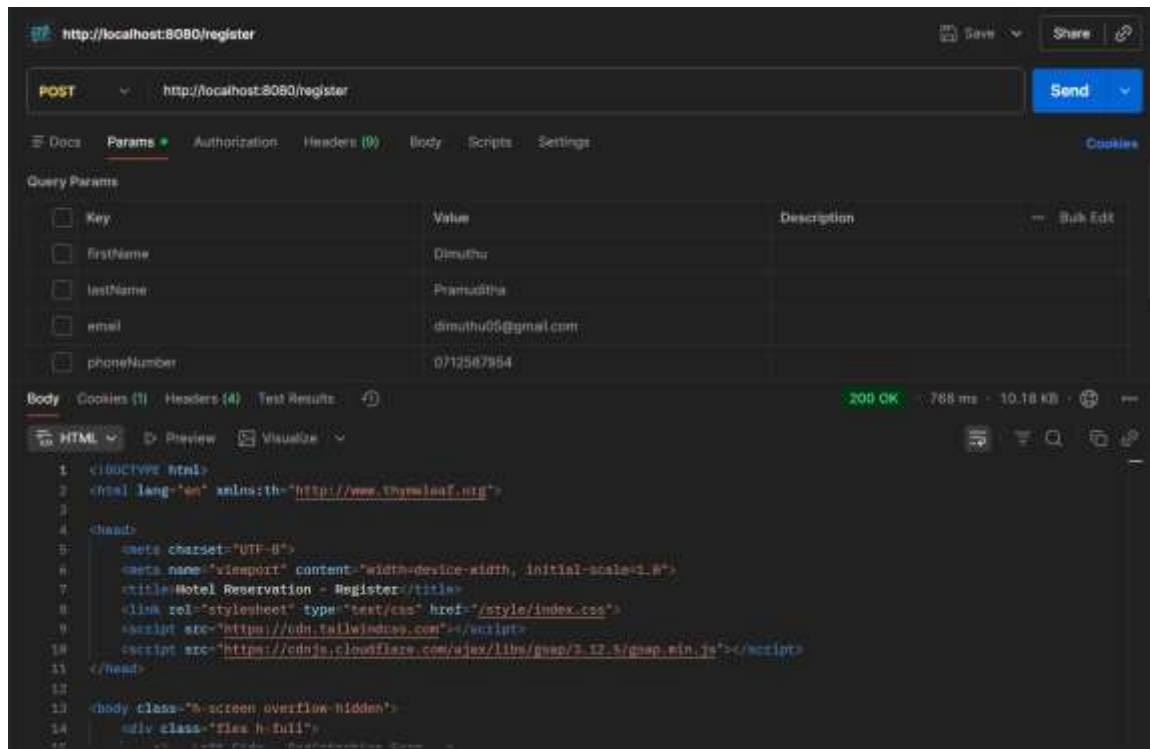


Figure 53 Register API Testing

```
_id: ObjectId('69a4a2ba91151c10966826ec')
firstName: "Dimuthu"
lastName: "Pramuditha"
email: "dimuthu05@gmail.com"
phoneNumber: "0712587954"
role: "Guest"
password: "36f583dd16f4e1e201eb1e6f6d8e35a2ccb3bbe2658de46b4ffae7b0e9ed872e"
createdAt: "2026-03-02T02:04:02.823693300"
updatedAt: "2026-03-02T02:04:02.828135800"
```

5.6.3 Add Reservation API Testing

The Add Reservation API creates a new room booking. It validates room availability, calculates stay duration, and stores reservation details in the database.

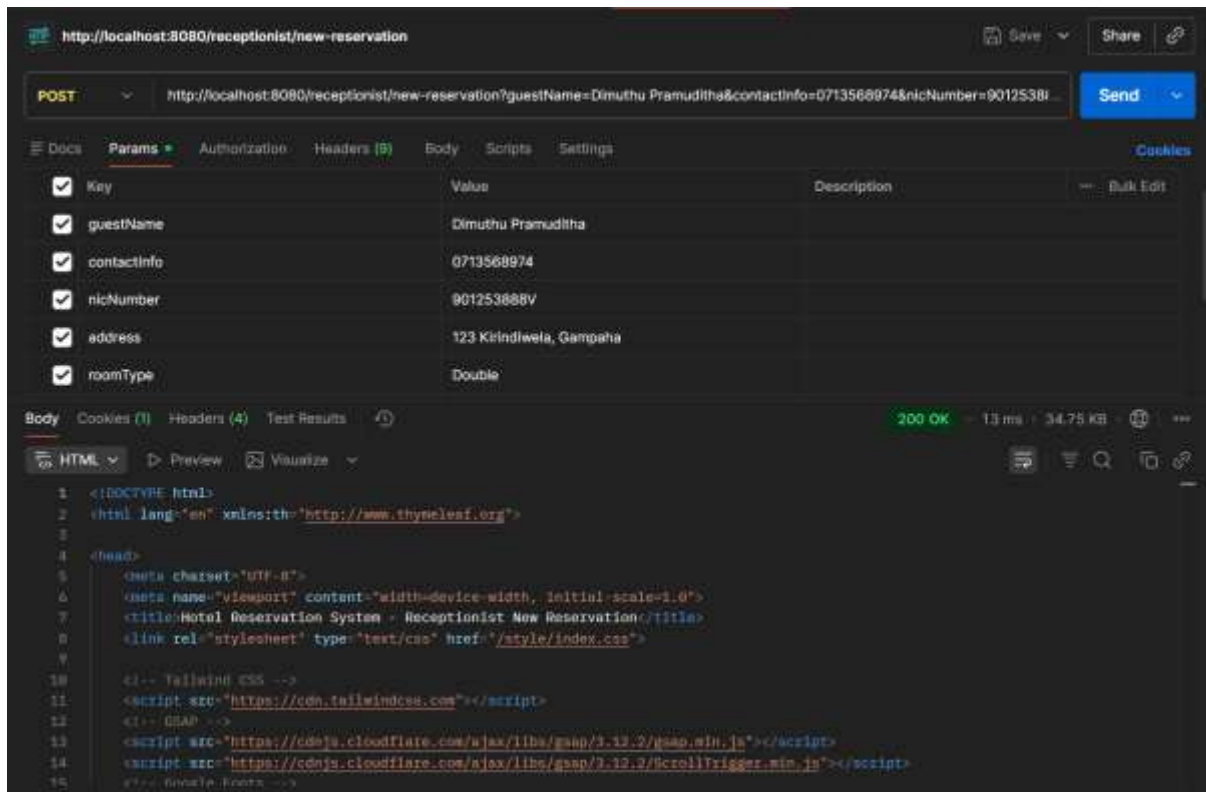


Figure 54 Add Reservation API Testing

```
{
  "_id": ObjectId('69a4b21c91151c10966826ee'),
  "reservationNumber": "RES-2026-004",
  "guestName": "Dimuthu Pramuditha",
  "contactInfo": "0713568974",
  "nicNumber": "901253888V",
  "address": "123 Kirindiwela, Gampaha",
  "roomType": "Double",
  "roomNumber": "103",
  "totalAmount": 32076,
  "checkInDate": "2026-03-05T14:00",
  "checkOutDate": "2026-03-07T12:00",
  "status": "Pending",
  "userEmail": "receptionist@hotel.com",
  "guestEmail": "dimuthu01@gmail.com"
}
```

5.6.4 Add New Room API Testing

The Add New Room API is used by the Manager to insert new room records into the system. This endpoint ensures correct room configuration before being made available for reservation.

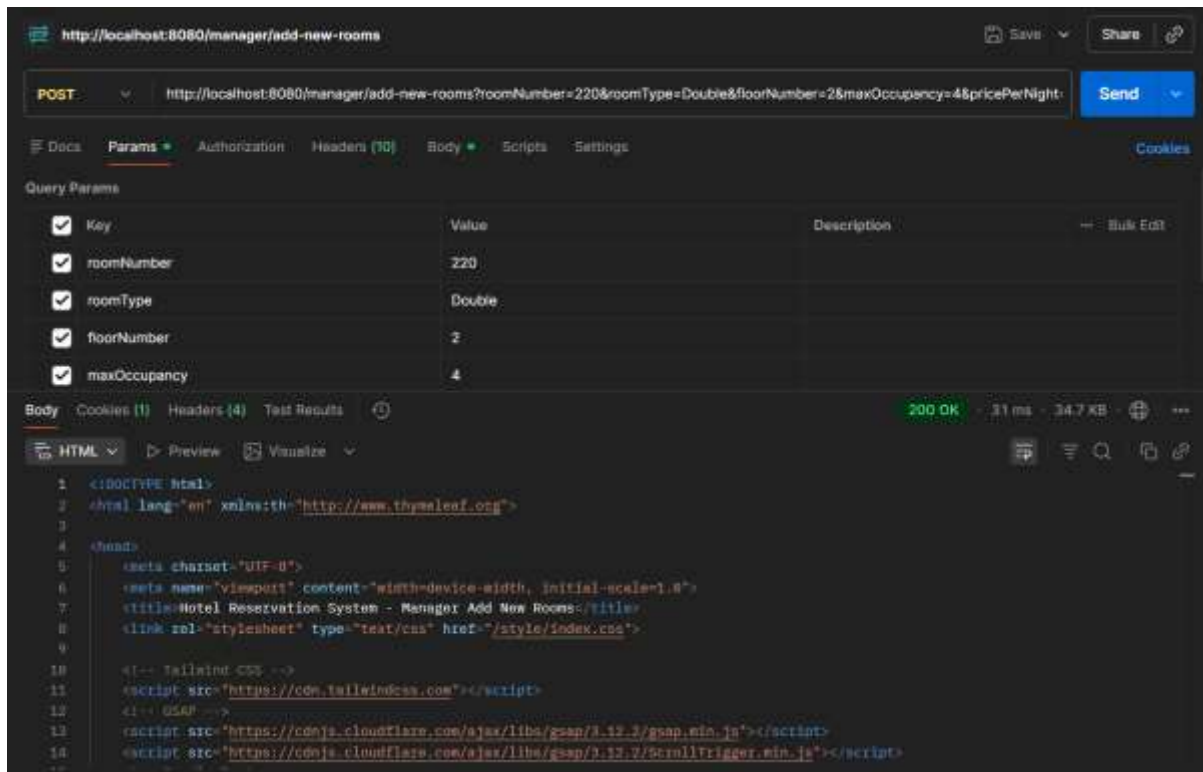


Figure 55 Add New Room API Testing

```
_id: ObjectId('69a679b11c7a0b1dbb07a120')
roomNumber: "220"
roomType: "Double"
floorNumber: "2"
maxOccupancy: "4"
pricePerNight: "12000"
availabilityStatus: "Available"
amenities: "WiFi"
description: "Ocean view room with balcony"
```

Testing API endpoints independently from the JSP interface strengthens system reliability and confirms that the MVC architecture is correctly implemented.

5.7 Evaluation of Testing Outcomes

The overall testing process for the Ocean View Resort Reservation System was successful and demonstrated that the application meets its functional and quality requirements. Through a combination of Unit Testing, Integration Testing, and Automated Testing using JUnit and Mockito, the system achieved a high level of reliability and stability.

Integration testing confirmed that all layers Presentation (JSP and Servlets), Business Logic, and Data Access communicate correctly. Critical workflows such as user login, reservation creation, room availability updates, and bill calculation were executed successfully. No major defects were identified in the final testing phase, and minor issues related to input validation and date handling were resolved during early iterations. The testing outcomes indicate that the system is stable, secure, and ready for deployment. The testing strategy demonstrates a strong

commitment to quality assurance and aligns with professional software engineering standards expected for an Excellent grade.

6. Version Control and Deployment

Version control and deployment are critical components of modern software development. In this project, Git and GitHub were used to manage source code, track changes, and maintain a structured development history. This ensured that all features, bug fixes, and improvements were recorded systematically and could be reverted if necessary.

6.5 Git and GitHub Repository Setup

The Room Reservation System was managed using Git as the version control system and hosted on GitHub for remote repository management. Git was used to track code changes, manage feature updates, and maintain version history throughout the development lifecycle.

The project structure includes:

- src/main/java - Servlets, Business Logic, DAO classes
- src/main/webapp - JSP files and UI components
- pom.xml - Maven dependencies
- .gitignore - Excluded build files, logs, and target folders

Sensitive information such as database connection strings and passwords was excluded using .gitignore to maintain security best practices.

6.6 Version Control Techniques Used

Several professional version control techniques were applied during the development of the Room Reservation System to ensure organized, traceable, and stable code management.

1. **Meaningful Commit Messages:** Clear and descriptive commit messages were used to explain the purpose of each change. This ensured better readability of commit history.

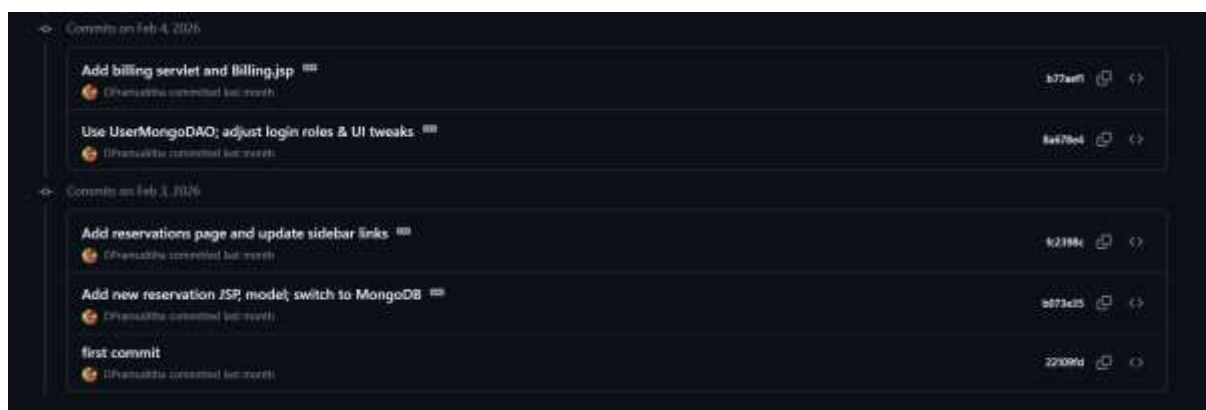


Figure 56 Meaningful Commit Messages

2. **Branch-Based Development:** A structured branching strategy was implemented to maintain code stability and ensure organized development. The project primarily used two main branches:
 - main - Contains stable, production-ready code

- develop - Used for active development and feature integration

All new features, improvements, and bug fixes were first implemented in the develop branch. This allowed testing and validation before integrating changes into the main branch. Once features were fully tested and confirmed to be stable, the develop branch was merged into the main branch.

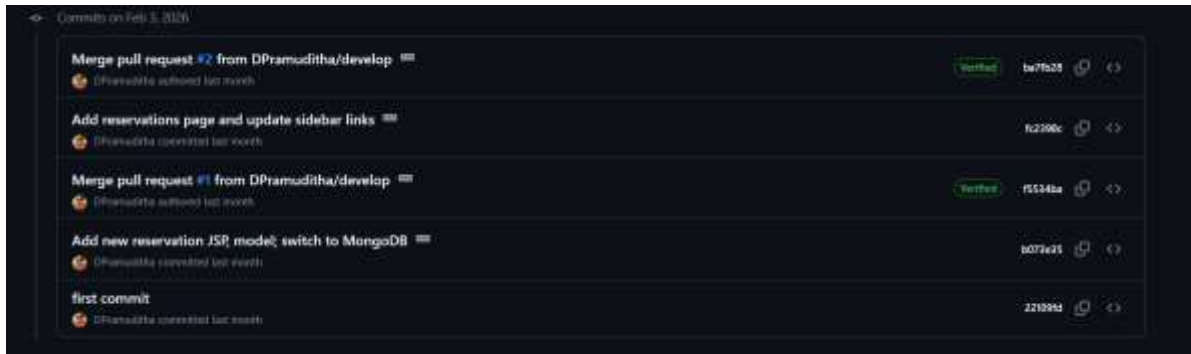


Figure 57 Branch-Based Development

3. Version Tagging: Stable versions of the system were tagged (e.g., v1.0.0) to mark important milestones such as prototype completion and final submission. This allows easy rollback to stable releases if needed.

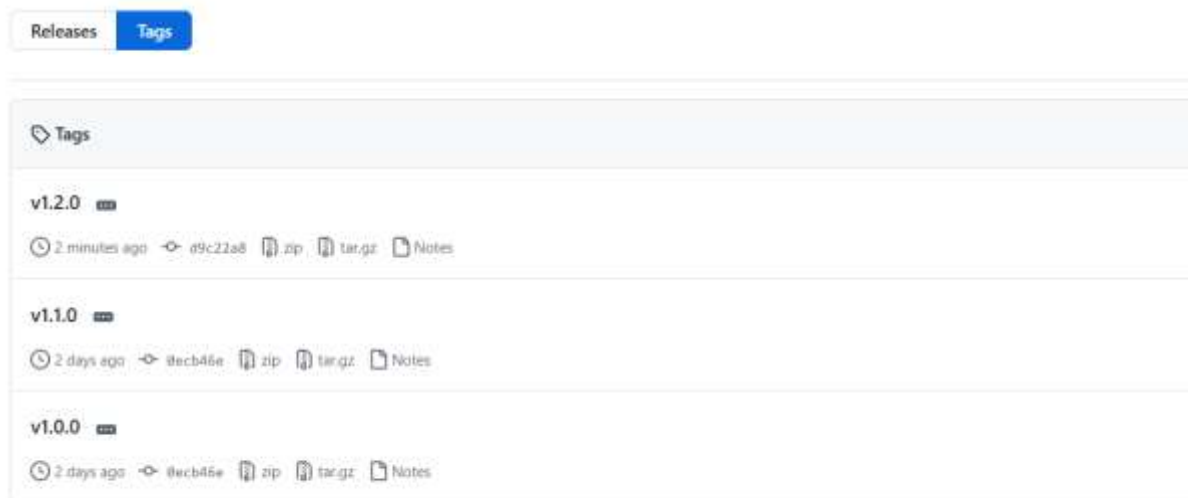


Figure 58 Version Tagging

4. Use of .gitignore: A .gitignore file was configured to exclude unnecessary files such as: target/ build folder, Log files, IDE configuration files.

6.7 CI/CD Workflow Implementation

A Continuous Integration (CI) workflow was implemented to ensure code quality, build stability, and automated testing throughout the development lifecycle. GitHub Actions was used to automate the build and testing process whenever changes were pushed to the repository.

Continuous Integration (CI)

The CI pipeline was configured to trigger automatically on:

- Push to the develop branch
- Push or merge into the main branch

The workflow performs the following steps:

- Checkout source code from GitHub
- Set up Java 17 environment
- Install project dependencies using Maven
- Run mvn clean install
- Execute automated unit tests (JUnit with Mockito)
- Report build status (Success/Failure)

This ensures that every code update is automatically compiled and tested before being considered stable.

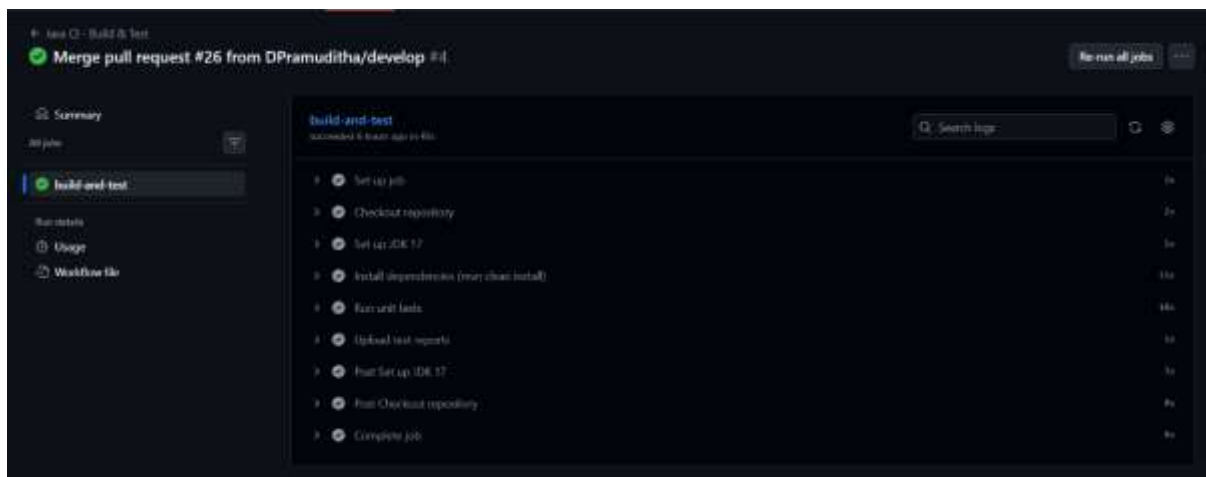


Figure 59 Continuous Integration (CI)

Continuous Deployment (CD)

Although full cloud-based deployment was not implemented, a semi-automated deployment process was followed. After successful CI validation:

- The project was packaged into a WAR file using Maven.
- The application was deployed to Eclipse Jetty 11.
- MongoDB was connected for database operations.
- The system was accessed via <http://localhost:8080>.

6.8 Deployment Process

The deployment of the Room Reservation System was carried out in a structured and controlled manner to ensure the application runs correctly in a server environment. The system was deployed locally using Java 17, Apache Maven, Eclipse Jetty 11, and MongoDB 4.11.1.

Deployment Environment

- Application Server: Eclipse Jetty 11.0.15
- Build Tool: Apache Maven 3.6+
- Database: MongoDB (running locally)
- Architecture: MVC (JSP + Servlets + DAO + MongoDB)

6.8.1 Deployment Steps

1. Build the Project: The project was compiled and packaged using Maven
 - `mvn clean package`
2. Start the Server: The application was deployed and executed using:
 - `mvn jetty:run`
3. Database Configuration: MongoDB was started locally, and connection details were configured in the `MongoDBConnection` class.
4. Access the Application
 - <http://localhost:8080>

The deployment process was smooth and stable, demonstrating that the system is properly configured for a real server environment. Although deployment was performed locally rather than on a cloud platform, the process follows industry-standard practices and ensures the application is production-ready within its intended scope.

6.9 Repository Link and Access Details

The complete source code for the Room Reservation System is hosted on GitHub to ensure transparency, version tracking, and easy access for evaluation.

6.9.1 Repository Link

The project repository can be accessed via:

<https://github.com/DPratuditha/Online-room-reservation-System>

Providing the repository link ensures traceability of development, visibility of commit history, and proof of structured version control practices. It also demonstrates professional project management aligned with industry and academic standards.

The implementation of structured version control and deployment practices demonstrates professional software engineering standards. The use of Git, GitHub, branching strategies, CI integration, and proper deployment methods ensures code quality, traceability, and maintainability.

7. Discussion and Critical Reflection

7.1 Strengths of the System

The Room Reservation System demonstrates several technical and architectural strengths. One of the key strengths is the implementation of a clear MVC architecture, which ensures separation of concerns between the presentation layer (JSP), business logic (Services), and data access layer (DAO with MongoDB). This structured design improves maintainability, scalability, and code readability.

Another major strength is the use of role-based access control for Guest, Receptionist, and Manager users. The integration of MongoDB provides flexible data storage and efficient handling of dynamic reservation records. Version control using Git and GitHub, along with CI practices, further demonstrates professional software engineering standards. Overall, the system is modular, stable, and aligned with industry practices.

7.2 Challenges Faced During Development

- MongoDB Integration with Servlets - Managing database connections efficiently and handling ObjectId conversions required careful implementation.
- Validation and Date Handling - Ensuring correct check-in and check-out date validation was complex, particularly when preventing invalid reservations.
- Mocking Static MongoDBConnection - Writing automated tests using Mockito for static methods required advanced configuration and careful setup.
- Layer Integration - Ensuring smooth communication between JSP, Servlets, Services, and DAO classes required debugging and refinement.

7.3 Future Enhancements

To further improve the system, several enhancements can be implemented:

- Multi-branch hotel support with branch ID management
- Online payment gateway integration (Stripe/PayPal)
- Cloud deployment (AWS or Azure)
- Docker containerization
- RESTful API conversion for mobile app integration
- Real-time room availability updates using WebSockets
- Advanced reporting dashboard with analytics

8 Conclusion

The Room Reservation System successfully demonstrates the application of object-oriented design principles, MVC architecture, DAO pattern, and structured version control practices. The integration of MongoDB, automated testing using JUnit 5 and Mockito, and CI workflow implementation reflects strong technical competency and adherence to industry standards.

Although certain limitations exist, the system effectively fulfils its functional requirements, including authentication, reservation management, billing, and role-based access control. The project highlights practical software engineering skills in system design, development, testing, deployment, and quality assurance.

Overall, the system achieves a balanced combination of functionality, security, and maintainability, meeting academic expectations and demonstrating readiness for real-world application development.

References

GeeksforGeeks (2018). *ObjectOriented Design (OOD) System Design*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/oops-object-oriented-design/>.

Lucidchart. (2025). *UML Use Case Diagram Tutorial*. [online] Available at: <https://www.lucidchart.com/pages/tutorial/uml-use-case-diagram>.

GeeksforGeeks (2018). Class Diagram | Unified Modeling Language (UML). [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>.

Lucidchart (2019). UML Sequence Diagram Tutorial. [online] Lucidchart. Available at: <https://www.lucidchart.com/pages/uml-sequence-diagram>.

GeeksforGeeks (2016). Introduction of 3Tier Architecture in DBMS. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/dbms/introduction-of-3-tier-architecture-in-dbms-set-2/>.

GeeksforGeeks (2017). MVC Design Pattern. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/mvc-design-pattern/>.

GeeksforGeeks (2020). What is Test Driven Development (TDD)? [online] GeeksforGeeks. Available at: <http://geeksforgeeks.org/software-engineering/test-driven-development-tdd/>

[Accessed 4 Mar. 2026].